

Frontend JavaScript Logic & Engines

Project: T&C; Red Flag Scanner | Team: CorruptX

File: frontend/js/chatbot.js

Size: 17162 bytes

```

1 | /**
2 |  * CorruptX - T&C AI Privacy Assistant (Chatbot)
3 |  * Intelligent NLP & Privacy Knowledge Engine (Offline + Backend Hybrid)
4 |  */
5 |
6 | (function () {
7 |     // Knowledge Base Patterns
8 |     const PRIVACY_KNOWLEDGE = [
9 |         {
10 |             keywords: ["arbitration", "class action", "court", "sue", "dispute", "lawsuit", "jury"],
11 |             title: "■ Mandatory Arbitration & Class Action Waivers",
12 |             reply: `<b>Mandatory Arbitration Clauses</b> strip away your constitutional right to take a company to
13 | <b>Why it's a Red Flag:</b>
14 | • Disputes are settled behind closed doors by private arbitrators often hired by the corporation.
15 | • You cannot band together with other affected users (Class Action Waiver).
16 | • It drastically lowers the company's liability for privacy breaches.<br><br>
17 | <i>Tip: Some companies allow you to "opt-out" of arbitration by sending a written notice within 30 days of openi
18 |     },
19 |     {
20 |         keywords: ["sell", "sold", "monetiz", "third party", "third-party", "partners", "advertis", "brokers", "sharing"],
21 |         title: "■ Data Selling & Third-Party Sharing",
22 |         reply: `<b>Data Selling & Sharing Clauses</b> permit companies to package your personal informatio
23 | <b>What to look out for:</b>
24 | • Phrases like <i>"We may share aggregated or anonymized data with third parties for marketing purposes."</i>
25 | • Phrases like <i>"Partners may receive device identifiers and demographic data."</i><br><br>
26 | <i>CorruptX Advice: Under CCPA (California) and GDPR (Europe), you have the legal right to click "Do Not Sell
27 |     },
28 |     {
29 |         keywords: ["track", "location", "gps", "biometric", "face", "fingerprint", "microphone", "camera", "listen", "reco
30 |         title: "■ Continuous Surveillance & Tracking",
31 |         reply: `<b>Surveillance & Tracking Clauses</b> allow apps to log your physical location, device se
32 | <b>Red Flag Triggers:</b>
33 | • Background GPS / Geofencing tracking.
34 | • Access to contact lists, audio/video data, or device fingerprinting.<br><br>
35 | <i>Defense: Always restrict background location permissions in your iOS/Android settings!</i>`
36 |     },
37 |     {
38 |         keywords: ["change", "modify", "update", "unilateral", "without notice", "discretion"],
39 |         title: "■ Unilateral Contract Modifications",
40 |         reply: `<b>Unilateral Modification Clauses</b> state that the company can change the terms at any time
41 | <b>Why it's Dangerous:</b>
42 | • A company could introduce data-selling or fee increases tomorrow, and your continued use of the app constitutes "agreee
43 | <i>Standard Best Practice: Reputable services notify users via email 30 days prior to major policy changes.</i>
44 |     },
45 |     {
46 |         keywords: ["delete", "deletion", "retain", "retention", "forever", "indefinitely", "store", "erase"],
47 |         title: "■ Account Deletion & Data Retention",
48 |         reply: `<b>Data Retention Clauses</b> specify how long a platform keeps your data after you delete you
49 | <b>Watch out for:</b>
50 | • <i>"We retain backup copies of user content indefinitely."</i>
51 | • <i>"Deleted accounts may take up to 90 days or longer to erase from servers."</i><br><br>
52 | <i>GDPR Right: Under GDPR Article 17 ("Right to Erasure"), platforms are legally mandated to erase personal da
53 |     },
54 |     {
55 |         keywords: ["copyright", "intellectual property", "license", "royalty-free", "own", "ownership", "photos", "content
56 |         title: "■ Content Rights & IP Ownership",
57 |         reply: `<b>IP Ownership Clauses</b> determine who owns the photos, text, or files you upload.<br>
58 | <b>Common Trap:</b> While policies claim "you retain ownership", hidden text grants them a <i>"worldwi
59 | <i>Risk: This allows companies to use your uploaded photos to train AI models or use them in ads without payin
60 |     },

```

```

61 |     {
62 |         keywords: ["gdpr", "ccpa", "right", "rights", "privacy law", "opt out", "opt-out", "erase", "access"],
63 |         title: "■ Digital Privacy Laws (GDPR & CCPA)",
64 |         reply: `<b>Your Legal Privacy Rights:</b><br><br>
65 | 1. <b>Right to Access:</b> Request a complete zip file of all data collected on you.<br>
66 | 2. <b>Right to Erasure / Forgotten:</b> Request full deletion of your profile.<br>
67 | 3. <b>Right to Opt-Out:</b> Require companies to cease selling/sharing data.<br>
68 | 4. <b>Right to Portability:</b> Transfer your data to another platform.`
69 |     },
70 |     {
71 |         keywords: ["score", "calculate", "how it works", "corruptx", "algorithm", "weight", "negation"],
72 |         title: "■ How CorruptX Risk Scoring Works",
73 |         reply: `<b>CorruptX Dual Engine Scoring:</b><br><br>
74 | • <b>Clause Segmentation:</b> Splits policies into individual semantic clauses.<br>
75 | • <b>Category Pattern Matching:</b> Scans against 14 cybersecurity threat categories (Arbitration, Data Sell
76 | • <b>Negation Protection:</b> Recognizes privacy promises like <i>"We DO NOT sell data"</i> so y
77 | • <b>Weighted Risk Score (0-100):</b> 0 = Maximum Privacy, 100 = Severe Privacy & Legal Violations.`
78 |     },
79 |     {
80 |         keywords: ["scan", "scanned", "my document", "my policy", "results", "red flag", "flagged"],
81 |         isScannedDocQuery: true
82 |     }
83 | ];
84 |
85 | // DOM Elements setup
86 | document.addEventListener("DOMContentLoaded", () => {
87 |     injectChatbotUI();
88 |     initChatbotEvents();
89 | });
90 |
91 | function injectChatbotUI() {
92 |     if (document.getElementById("tc-chatbot-widget")) return;
93 |
94 |     const widgetHtml = `
95 |         <div id="tc-chatbot-widget">
96 |             <!-- Floating Trigger Button -->
97 |             <button id="tc-chatbot-toggle" title="Open T&C AI Privacy Assistant">
98 |                 <div class="chatbot-toggle-icon">
99 |                     <svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="cu
100 |                 </div>
101 |                 <span class="chatbot-toggle-text">AI Privacy Advisor</span>
102 |                 <span class="chatbot-pulse-dot"></span>
103 |             </button>
104 |
105 |             <!-- Chat Panel Window -->
106 |             <div id="tc-chatbot-modal" class="chatbot-modal hidden">
107 |                 <!-- Header -->
108 |                 <div class="chatbot-header">
109 |                     <div class="chatbot-header-info">
110 |                         <div class="chatbot-avatar">■</div>
111 |                         <div>
112 |                             <div class="chatbot-title">CorruptX Privacy Assistant</div>
113 |                             <div class="chatbot-status"><span class="status-indicator-dot"></span> Online • NLP Cy
114 |                         </div>
115 |                     </div>
116 |                     <div class="chatbot-header-actions">
117 |                         <button id="chatbot-clear-btn" title="Clear Chat History">■</button>
118 |                         <button id="chatbot-close-btn" title="Close Panel">X</button>
119 |                     </div>
120 |                 </div>
121 |
122 |                 <!-- Body / Messages -->
123 |                 <div id="chatbot-messages" class="chatbot-messages">
124 |                     <div class="chat-message assistant">
125 |                         <div class="message-content">
126 |                             ■ Hello! I am your <b>CorruptX AI Privacy Assistant</b>.<br><br>
127 |                             Ask me anything about Terms & Conditions, privacy rights, data selling, forced arbitration, or your
128 |                         </div>
129 |                     </div>

```

```

130 |         </div>
131 |
132 |         <!-- Suggestion Chips -->
133 |         <div id="chatbot-chips" class="chatbot-chips">
134 |             <button class="chip-btn" data-query="Does my scanned policy sell data?">■ Analyze my scanned policy</button>
135 |             <button class="chip-btn" data-query="What is forced arbitration?">■■ Forced Arbitration</button>
136 |             <button class="chip-btn" data-query="Is my data sold to third parties?">■ Data Selling</button>
137 |             <button class="chip-btn" data-query="What are my GDPR rights?">■■ GDPR Rights</button>
138 |         </div>
139 |
140 |         <!-- Input Footer -->
141 |         <div class="chatbot-footer">
142 |             <input type="text" id="chatbot-input" placeholder="Ask a question about T&C clauses..." autocomplete="off">
143 |             <button id="chatbot-send-btn" title="Send Message">
144 |                 <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="black">
145 |                     </svg>
146 |             </button>
147 |         </div>
148 |     </div>
149 | `;
150 |
151 |     document.body.insertAdjacentHTML("beforeend", widgetHtml);
152 | }
153 |
154 | function initChatbotEvents() {
155 |     const toggleBtn = document.getElementById("tc-chatbot-toggle");
156 |     const closeBtn = document.getElementById("chatbot-close-btn");
157 |     const clearBtn = document.getElementById("chatbot-clear-btn");
158 |     const modal = document.getElementById("tc-chatbot-modal");
159 |     const sendBtn = document.getElementById("chatbot-send-btn");
160 |     const inputEl = document.getElementById("chatbot-input");
161 |     const chipsContainer = document.getElementById("chatbot-chips");
162 |
163 |     if (!toggleBtn || !modal) return;
164 |
165 |     toggleBtn.addEventListener("click", () => {
166 |         modal.classList.toggle("hidden");
167 |         if (!modal.classList.contains("hidden")) {
168 |             inputEl.focus();
169 |             scrollToBottom();
170 |         }
171 |     });
172 |
173 |     closeBtn.addEventListener("click", () => {
174 |         modal.classList.add("hidden");
175 |     });
176 |
177 |     clearBtn.addEventListener("click", () => {
178 |         const messagesEl = document.getElementById("chatbot-messages");
179 |         messagesEl.innerHTML = `
180 |             <div class="chat-message assistant">
181 |                 <div class="message-content">
182 |                     Conversation cleared. What privacy or T&C topic would you like to explore?
183 |                 </div>
184 |             </div>
185 |         `;
186 |     });
187 |
188 |     sendBtn.addEventListener("click", () => handleUserSend());
189 |
190 |     inputEl.addEventListener("keydown", (e) => {
191 |         if (e.key === "Enter") {
192 |             e.preventDefault();
193 |             handleUserSend();
194 |         }
195 |     });
196 |
197 |     if (chipsContainer) {
198 |         chipsContainer.addEventListener("click", (e) => {

```

```

199 |     const chip = e.target.closest(".chip-btn");
200 |     if (chip) {
201 |         const query = chip.getAttribute("data-query");
202 |         if (query) {
203 |             inputEl.value = query;
204 |             handleUserSend();
205 |         }
206 |     }
207 | });
208 | }
209 | }
210 |
211 | async function handleUserSend() {
212 |     const inputEl = document.getElementById("chatbot-input");
213 |     const text = inputEl.value ? inputEl.value.trim() : "";
214 |     if (!text) return;
215 |
216 |     // Append user message
217 |     appendMessage("user", text);
218 |     inputEl.value = "";
219 |
220 |     // Show typing indicator
221 |     const typingId = appendTypingIndicator();
222 |     scrollToBottom();
223 |
224 |     // Small natural delay
225 |     await sleep(400);
226 |
227 |     // Try backend AI endpoint first if online, else client engine
228 |     let responseHtml = "";
229 |     if (window.AppState && window.AppState.isBackendAvailable) {
230 |         try {
231 |             const activeScan = window.getActiveScan ? window.getActiveScan() : null;
232 |             const res = await fetch((window.AppState.apiBase || "") + "/api/chat", {
233 |                 method: "POST",
234 |                 headers: { "Content-Type": "application/json" },
235 |                 body: JSON.stringify({ message: text, active_scan: activeScan })
236 |             });
237 |             if (res.ok) {
238 |                 const data = await res.json();
239 |                 responseHtml = data.reply;
240 |             }
241 |         } catch (e) {
242 |             console.warn("Backend chat endpoint unavailable, falling back to offline assistant:", e);
243 |         }
244 |     }
245 |
246 |     if (!responseHtml) {
247 |         responseHtml = generateOfflineResponse(text);
248 |     }
249 |
250 |     removeTypingIndicator(typingId);
251 |     appendMessage("assistant", responseHtml);
252 |     scrollToBottom();
253 | }
254 |
255 | function generateOfflineResponse(query) {
256 |     const qLower = query.toLowerCase();
257 |     const activeScan = window.getActiveScan ? window.getActiveScan() : null;
258 |
259 |     // Check if query is about the scanned document
260 |     const isDocQuery = qLower.includes("my policy") || qLower.includes("scanned") || qLower.includes("my document") || qLower.includes("document");
261 |
262 |     if (isDocQuery) {
263 |         if (!activeScan) {
264 |             return `<b>No active document scan found.</b><br>Please paste a policy into the <a href="scanned">scanned</a> chip.`;
265 |         }
266 |
267 |         const score = activeScan.risk_score || 0;

```

```

268 |     const level = activeScan.risk_level || "SAFE";
269 |     const docName = activeScan.document_name || "Uploaded Policy";
270 |     const redFlagsCount = activeScan.red_flags || 0;
271 |     const flagged = activeScan.flagged_clauses || [];
272 |
273 |     if (qLower.includes("sell") || qLower.includes("sharing") || qLower.includes("third party")) {
274 |         const dataSellingFlag = flagged.find(c => (c.matches || []).some(m => m.category_id === "DATA_SELLING" ||
275 |             if (dataSellingFlag) {
276 |                 return `■■ <b>Data Selling Detected in "${docName}":</b><br><br>
277 | Line snippet: <i>${(dataSellingFlag.text || "").substring(0, 150)}...</i><br><br>
278 | This policy explicitly permits sharing or monetizing user data with third parties.`;
279 |             } else {
280 |                 return `■ <b>No explicit Data Selling clauses</b> were flagged in your scan of <b>"${docName}"</b>.`;
281 |             }
282 |         }
283 |
284 |         if (qLower.includes("arbitration") || qLower.includes("court") || qLower.includes("sue")) {
285 |             const arbFlag = flagged.find(c => (c.matches || []).some(m => m.category_id === "ARBITRATION_DISPUTES"));
286 |             if (arbFlag) {
287 |                 return `■■ <b>Forced Arbitration Clause Found in "${docName}":</b><br><br>
288 | Snippet: <i>${(arbFlag.text || "").substring(0, 150)}...</i><br><br>
289 | You waive your right to sue in court or join class actions under this contract.`;
290 |             } else {
291 |                 return `■ <b>No Mandatory Arbitration clauses</b> were detected in <b>"${docName}"</b>.`;
292 |             }
293 |         }
294 |
295 |         // Default scan summary response
296 |         let flagsList = flagged.slice(0, 3).map(f => `• <b>${f.matches[0]?.category_name || "Risk"}:</b> <b>${f.matches[0]?.text}</b>`);
297 |         return `■ <b>Summary for "${docName}":</b><br><br>
298 | • <b>Risk Score:</b> ${score}/100 (<span style="color:${level==='HIGH RISK'?`var(--accent-crimson)`:`var(--accent-cyan)`}</span>);
299 | • <b>Red Flags Detected:</b> ${redFlagsCount}<br><br>
300 | ${flagsList ? "<b>Top Highlighted Flags:</b><br>" + flagsList : "No critical red flags detected."}<br>
301 | <a href="results.html" style="color:var(--accent-cyan); font-weight:bold;">View full interactive report <b>■■</b></a>`
302 |     }
303 |
304 |     // Knowledge base matching
305 |     for (const item of PRIVACY_KNOWLEDGE) {
306 |         if (item.isScannedDocQuery) continue;
307 |         if (item.keywords.some(kw => qLower.includes(kw))) {
308 |             return item.reply;
309 |         }
310 |     }
311 |
312 |     // Default Fallback
313 |     return `I evaluated your query about <b>"${query}"</b>.<br><br>
314 | Here are key privacy principles to keep in mind:<br>
315 | 1. <b>Always check for Mandatory Arbitration</b> clauses before accepting.<br>
316 | 2. <b>Look out for "Data Sharing"</b> with ambiguous "partners".<br>
317 | 3. <b>Check if terms change without notice.</b><br>
318 | You can paste any policy into the <a href="scanner.html" style="color:var(--accent-cyan);">CorruptX Scanner</a>`
319 | }
320 |
321 | function appendMessage(sender, htmlContent) {
322 |     const messagesEl = document.getElementById("chatbot-messages");
323 |     if (!messagesEl) return;
324 |
325 |     const msgDiv = document.createElement("div");
326 |     msgDiv.className = `chat-message ${sender}`;
327 |     msgDiv.innerHTML = `<div class="message-content">${htmlContent}</div>`;
328 |     messagesEl.appendChild(msgDiv);
329 | }
330 |
331 | function appendTypingIndicator() {
332 |     const messagesEl = document.getElementById("chatbot-messages");
333 |     if (!messagesEl) return null;
334 |
335 |     const id = "typing_" + Date.now();
336 |     const typingDiv = document.createElement("div");

```

```
337 |     typingDiv.id = id;
338 |     typingDiv.className = "chat-message assistant typing";
339 |     typingDiv.innerHTML = `
340 |         &lt;div class="message-content"&gt;
341 |             &lt;span class="typing-dot"&gt;&lt;/span&gt;
342 |             &lt;span class="typing-dot"&gt;&lt;/span&gt;
343 |             &lt;span class="typing-dot"&gt;&lt;/span&gt;
344 |         &lt;/div&gt;
345 |     `;
346 |     messagesEl.appendChild(typingDiv);
347 |     return id;
348 | }
349 |
350 | function removeTypingIndicator(id) {
351 |     if (!id) return;
352 |     const el = document.getElementById(id);
353 |     if (el) el.remove();
354 | }
355 |
356 | function scrollToBottom() {
357 |     const messagesEl = document.getElementById("chatbot-messages");
358 |     if (messagesEl) {
359 |         messagesEl.scrollTop = messagesEl.scrollHeight;
360 |     }
361 | }
362 |
363 | function sleep(ms) {
364 |     return new Promise(resolve =&gt; setTimeout(resolve, ms));
365 | }
366 | }());
```

File: frontend/js/client_analyzer.js

Size: 12789 bytes

```

1 | /**
2 |  * CorruptX T&C Red Flag Scanner - Client-Side Analysis Engine
3 |  * 100% Offline Standalone Architecture
4 |  * Provides identical NLP rule processing and scoring when running in pure browser mode.
5 |  */
6 |
7 | class ClientClauseAnalyzer {
8 |   constructor(rulesData) {
9 |     this.rulesData = rulesData || window.TC_RISK_RULES || { categories: [] };
10 |    this.categories = this.rulesData.categories || [];
11 |    this.compiledRules = [];
12 |    this.initRules();
13 |  }
14 |
15 |   initRules() {
16 |     this.compiledRules = this.categories.map(cat => {
17 |       const compiledPatterns = [];
18 |       (cat.patterns || []).forEach(patternStr => {
19 |         try {
20 |           compiledPatterns.push(new RegExp(patternStr, "i"));
21 |         } catch (e) {
22 |           console.warn("Regex compile failed:", patternStr, e);
23 |         }
24 |       });
25 |
26 |       return {
27 |         id: cat.id,
28 |         name: cat.name,
29 |         severity: cat.severity || "LOW",
30 |         weight: cat.weight || 5,
31 |         color: cat.color || "#38bdf8",
32 |         explanation: cat.explanation || "",
33 |         why_it_matters: cat.why_it_matters || "",
34 |         recommendation: cat.recommendation || "",
35 |         keywords: (cat.keywords || []).map(k => k.toLowerCase()),
36 |         patterns: compiledPatterns
37 |       };
38 |     });
39 |   }
40 |
41 |   cleanText(rawText) {
42 |     if (!rawText) return "";
43 |     let cleaned = rawText
44 |       .replace(/["']/g, "'")
45 |       .replace(/['"]/g, '"')
46 |       .replace(/[\-\_]/g, " - ")
47 |       .replace(/\r\n/g, "\n")
48 |       .replace(/\r/g, "\n");
49 |     return cleaned.trim();
50 |   }
51 |
52 |   segmentClauses(text) {
53 |     const cleaned = this.cleanText(text);
54 |     if (!cleaned) return [];
55 |
56 |     // Mask common abbreviations
57 |     const abbrMap = [
58 |       { regex: /\be\.g\.b/gi, rep: "EG_TEMP_MARKER" },
59 |       { regex: /\bi\.e\.b/gi, rep: "IE_TEMP_MARKER" },
60 |       { regex: /\bU\.S\.b/gi, rep: "US_TEMP_MARKER" },
61 |       { regex: /\betc\.b/gi, rep: "ETC_TEMP_MARKER" },
62 |       { regex: /\bInc\.b/gi, rep: "INC_TEMP_MARKER" },
63 |       { regex: /\bLtd\.b/gi, rep: "LTD_TEMP_MARKER" },
64 |       { regex: /\bLLC\.b/gi, rep: "LLC_TEMP_MARKER" },
65 |       { regex: /\bSec\.b/gi, rep: "SEC_TEMP_MARKER" },
66 |       { regex: /\bNo\.b/gi, rep: "NO_TEMP_MARKER" },
67 |       { regex: /\bv\.b/gi, rep: "V_TEMP_MARKER" },

```

```

68 |     { regex: /\bvs\b/gi, rep: "VS_TEMP_MARKER" }
69 | ];
70 |
71 | let masked = cleaned;
72 | abbrMap.forEach(item => {
73 |     masked = masked.replace(item.regex, item.rep);
74 | });
75 |
76 | // Protect section numbering
77 | masked = masked.replace(/(\d)\.(\d)/g, "$1DECIMAL_DOT_MARKER$2");
78 |
79 | // Split on paragraphs or sentence punctuation followed by capital/digit or list headers
80 | const chunks = masked.split(/(?:\n\s*\n|(?<=[.!?])\s+(?=[A-Z0-9"])|(?<=[.!?])\s*(?=[0-9]+\.\s|[A-Z]\.\s|\\([a-z0-9
81 |
82 | const clauses = [];
83 | chunks.forEach(chunk => {
84 |     let restored = chunk;
85 |     restored = restored.replace(/EG_TEMP_MARKER/g, "e.g.");
86 |     restored = restored.replace(/IE_TEMP_MARKER/g, "i.e.");
87 |     restored = restored.replace(/US_TEMP_MARKER/g, "U.S.");
88 |     restored = restored.replace(/ETC_TEMP_MARKER/g, "etc.");
89 |     restored = restored.replace(/INC_TEMP_MARKER/g, "Inc.");
90 |     restored = restored.replace(/LTD_TEMP_MARKER/g, "Ltd.");
91 |     restored = restored.replace(/LLC_TEMP_MARKER/g, "LLC.");
92 |     restored = restored.replace(/SEC_TEMP_MARKER/g, "Sec.");
93 |     restored = restored.replace(/NO_TEMP_MARKER/g, "No.");
94 |     restored = restored.replace(/V_TEMP_MARKER/g, "v.");
95 |     restored = restored.replace(/VS_TEMP_MARKER/g, "vs.");
96 |     restored = restored.replace(/DECIMAL_DOT_MARKER/g, ".");
97 |     restored = restored.replace(/\s+/g, " ").trim();
98 |
99 |     if (restored.length >= 12 || /[a-zA-Z]{3,}/.test(restored)) {
100 |         clauses.push(restored);
101 |     }
102 | });
103 |
104 | return clauses.length > 0 ? clauses : (cleaned ? [cleaned] : []);
105 | }
106 |
107 | isNegatedProtection(clauseText, categoryId) {
108 |     const clauseLower = clauseText.toLowerCase();
109 |
110 |     const negationRules = {
111 |         data_sharing: [
112 |             /(?:we|company|our\s+service)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+(?:sell|share|disclose|rent|monetize|
113 |             /no\s+(?:third-party\s+sale|selling|sharing\s+with\s+third\s+parties)/,
114 |             /(?:we\s+do\s+not|never)\s+disclose\s+personal\s+data/
115 |         ],
116 |         tracking: [
117 |             /(?:we|company|our\s+service)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+(?:track|use\s+tracking|record\s+keys
118 |             /(?:no|zero)\s+(?:tracking|fingerprinting|telemetry|session\s+replay)/,
119 |             /do\s+not\s+use\s+(?:tracking\s+cookies|device\s+fingerprinting|analytics\s+pixels)/
120 |         ],
121 |         data_collection: [
122 |             /(?:we|company)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+collect\s+(?:biometric|location|sensitive|contacts|
123 |             /do\s+not\s+collect\s+location\s+data/
124 |         ],
125 |         location_tracking: [
126 |             /(?:we|company)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+(?:collect|track)\s+(?:your\s+)?location/,
127 |             /do\s+not\s+collect\s+location\s+data/
128 |         ],
129 |         advertising_profiling: [
130 |             /no\s+advertising\s+networks\s+are\s+embedded/,
131 |             /(?:we|company)\s+(?:do\s+not|will\s+not|never)\s+(?:serve\s+targeted|profile|sell\s+ads)/
132 |         ],
133 |         arbitration: [
134 |             /(?:we|company)\s+(?:do\s+not|will\s+not)\s+enforce\s+(?:mandatory\s+)?(?:binding\s+)?arbitration/,
135 |             /do\s+not\s+enforce\s+mandatory\s+binding\s+arbitration/
136 |         ]

```

```

137 |     };
138 |
139 |     const rules = negationRules[categoryId] || [];
140 |     for (let r of rules) {
141 |         if (r.test(clauseLower)) return true;
142 |     }
143 |     return false;
144 | }
145 |
146 | analyzeClause(clauseText, index) {
147 |     const clauseLower = clauseText.toLowerCase();
148 |     const matchedCategories = [];
149 |
150 |     for (const rule of this.compiledRules) {
151 |         if (this.isNegatedProtection(clauseText, rule.id)) {
152 |             continue;
153 |         }
154 |
155 |         let matchedFlag = false;
156 |         let matchedSnippet = null;
157 |
158 |         for (const pattern of rule.patterns) {
159 |             const m = clauseText.match(pattern);
160 |             if (m) {
161 |                 matchedFlag = true;
162 |                 matchedSnippet = m[0];
163 |                 break;
164 |             }
165 |         }
166 |
167 |         if (!matchedFlag) {
168 |             for (const kw of rule.keywords) {
169 |                 if (clauseLower.includes(kw)) {
170 |                     matchedFlag = true;
171 |                     matchedSnippet = kw;
172 |                     break;
173 |                 }
174 |             }
175 |         }
176 |
177 |         if (matchedFlag) {
178 |             matchedCategories.push({
179 |                 category_id: rule.id,
180 |                 category_name: rule.name,
181 |                 severity: rule.severity,
182 |                 weight: rule.weight,
183 |                 color: rule.color,
184 |                 explanation: rule.explanation,
185 |                 why_it_matters: rule.why_it_matters,
186 |                 recommendation: rule.recommendation,
187 |                 snippet: matchedSnippet
188 |             });
189 |         }
190 |     }
191 |
192 |     const rank = { HIGH: 3, MEDIUM: 2, LOW: 1 };
193 |
194 |     if (matchedCategories.length > 0) {
195 |         matchedCategories.sort((a, b) => {
196 |             const rDiff = (rank[b.severity] || 0) - (rank[a.severity] || 0);
197 |             return rDiff !== 0 ? rDiff : b.weight - a.weight;
198 |         });
199 |
200 |         const top = matchedCategories[0];
201 |         return {
202 |             index: index,
203 |             text: clauseText,
204 |             is_flagged: true,
205 |             category_id: top.category_id,

```

```

206 |         category_name: top.category_name,
207 |         severity: top.severity,
208 |         weight: top.weight,
209 |         color: top.color,
210 |         explanation: top.explanation,
211 |         why_it_matters: top.why_it_matters,
212 |         recommendation: top.recommendation,
213 |         all_matched_categories: matchedCategories.map(m => m.category_name)
214 |     };
215 | } else {
216 |     return {
217 |         index: index,
218 |         text: clauseText,
219 |         is_flagged: false,
220 |         category_id: null,
221 |         category_name: "Standard Clause",
222 |         severity: "SAFE",
223 |         weight: 0,
224 |         color: "#00ff88",
225 |         explanation: "No elevated risk indicators detected in this clause.",
226 |         why_it_matters: "Standard service provision or routine operational statement.",
227 |         recommendation: "Clause appears customary; verify context if unsure.",
228 |         all_matched_categories: []
229 |     };
230 | }
231 | }
232 |
233 | calculateRiskScore(totalClauses, flaggedClauses) {
234 |     if (totalClauses <= 0 || flaggedClauses.length === 0) {
235 |         return 5;
236 |     }
237 |
238 |     const highCount = flaggedClauses.filter(c => c.severity === "HIGH").length;
239 |     const medCount = flaggedClauses.filter(c => c.severity === "MEDIUM").length;
240 |     const lowCount = flaggedClauses.filter(c => c.severity === "LOW").length;
241 |
242 |     const weightedSum = (highCount * 16) + (medCount * 8) + (lowCount * 4);
243 |     const density = flaggedClauses.length / Math.max(totalClauses, 1);
244 |     const densityBonus = Math.min(20, Math.floor(density * 35));
245 |
246 |     let rawScore = Math.floor((weightedSum * 0.82) + densityBonus);
247 |
248 |     if (highCount >= 3) {
249 |         rawScore = Math.max(rawScore, 68);
250 |     } else if (highCount >= 1) {
251 |         rawScore = Math.max(rawScore, 42);
252 |     } else if (medCount >= 2) {
253 |         rawScore = Math.max(rawScore, 35);
254 |     }
255 |
256 |     return Math.max(0, Math.min(100, rawScore));
257 | }
258 |
259 | determineRiskLevel(score) {
260 |     if (score <= 30) return "SAFE";
261 |     if (score <= 60) return "CAUTION";
262 |     return "HIGH RISK";
263 | }
264 |
265 | scan(text, docName = "Submitted Policy") {
266 |     if (!text || text.trim().length === 0) {
267 |         return {
268 |             error: "Input text is empty. Please paste or upload a policy to scan.",
269 |             risk_score: 0,
270 |             risk_level: "SAFE",
271 |             clauses_analyzed: 0,
272 |             red_flags: 0
273 |         };
274 |     }

```

```

275 |
276 |     const clauses = this.segmentClauses(text);
277 |     const totalClauses = clauses.length;
278 |     const analyzedClauses = [];
279 |     const flaggedClauses = [];
280 |     const categoryCounts = {};
281 |
282 |     clauses.forEach((cl, idx) => {
283 |         const res = this.analyzeClause(cl, idx);
284 |         analyzedClauses.append ? analyzedClauses.append(res) : analyzedClauses.push(res);
285 |         if (res.is_flagged) {
286 |             flaggedClauses.push(res);
287 |             const catId = res.category_id;
288 |             if (!categoryCounts[catId]) {
289 |                 categoryCounts[catId] = {
290 |                     category_id: catId,
291 |                     category_name: res.category_name,
292 |                     severity: res.severity,
293 |                     color: res.color,
294 |                     count: 0,
295 |                     clauses: []
296 |                 };
297 |             }
298 |             categoryCounts[catId].count++;
299 |             categoryCounts[catId].clauses.push(res.index);
300 |         }
301 |     });
302 |
303 |     const score = this.calculateRiskScore(totalClauses, flaggedClauses);
304 |     const level = this.determineRiskLevel(score);
305 |
306 |     const severityCounts = {
307 |         HIGH: flaggedClauses.filter(c => c.severity === "HIGH").length,
308 |         MEDIUM: flaggedClauses.filter(c => c.severity === "MEDIUM").length,
309 |         LOW: flaggedClauses.filter(c => c.severity === "LOW").length
310 |     };
311 |
312 |     const sortedCategories = Object.values(categoryCounts).sort((a, b) => {
313 |         const diff = b.count - a.count;
314 |         if (diff !== 0) return diff;
315 |         const rank = { HIGH: 3, MEDIUM: 2, LOW: 1 };
316 |         return (rank[b.severity] || 0) - (rank[a.severity] || 0);
317 |     });
318 |
319 |     const summary = this.generateSummary(score, level, severityCounts, totalClauses);
320 |
321 |     return {
322 |         document_name: docName,
323 |         analyzed_at: new Date().toISOString(),
324 |         risk_score: score,
325 |         risk_level: level,
326 |         summary: summary,
327 |         clauses_analyzed: totalClauses,
328 |         red_flags: flaggedClauses.length,
329 |         severity_counts: severityCounts,
330 |         categories: sortedCategories,
331 |         flagged_clauses: flaggedClauses,
332 |         all_clauses: analyzedClauses,
333 |         engine_mode: "client-offline",
334 |         disclaimer: "This tool provides automated cybersecurity risk analysis for informational purposes and is not a subs
335 |     };
336 | }
337 |
338 | generateSummary(score, level, severities, totalClauses) {
339 |     const high = severities.HIGH || 0;
340 |     const med = severities.MEDIUM || 0;
341 |     const low = severities.LOW || 0;
342 |     const totalFlags = high + med + low;
343 |

```

```
344 |     if (level === "HIGH RISK") {
345 |         return `CRITICAL PRIVACY & LEGAL EXPOSURE DETECTED. Analysis identified ${totalFlags} red flags (${high} High-
346 |     } else if (level === "CAUTION") {
347 |         return `MODERATE USER EXPOSURE DETECTED. Identified ${totalFlags} flagged clauses (${high} High, ${med} Medium) ou
348 |     } else {
349 |         return `ACCEPTABLE RISK PROFILE. Minimal legal or data exposure detected across ${totalClauses} clauses. Only ${to
350 |     }
351 | }
352 | }
353 |
354 | // Global client-side scanner instance
355 | window.clientClauseAnalyzer = new ClientClauseAnalyzer(window.TC_RISK_RULES);
```

File: frontend/js/rules_data.js

Size: 20648 bytes

```

1 | /**
2 |  * CorruptX T&C Risk Taxonomy - Client-Side Rule Definitions
3 |  * Bundled for 100% true offline standalone capability.
4 |  */
5 | window.TC_RISK_RULES = {
6 |   "version": "1.0.0",
7 |   "taxonomy_name": "CorruptX CyberSecurity T&C Risk Taxonomy",
8 |   "categories": [
9 |     {
10 |      "id": "data_sharing",
11 |      "name": "Data Sharing",
12 |      "severity": "HIGH",
13 |      "weight": 15,
14 |      "color": "#ff3366",
15 |      "description": "Sharing, selling, transferring, or disclosing personal data to external third parties, business pa
16 |      "patterns": [
17 |        "share(?:s|d|ing)?\\s+(?:your|user|personal|sensitive)\\s+(?:information|data|details)\\s+with\\s+third\\s+parti
18 |        "disclose(?:s|d|ing)?\\s+(?:your|personal|user)\\s+(?:information|data)\\s+to\\s+(?:third\\s+parties|affiliates|
19 |        "sell(?:s|ing)?\\s+(?:or\\s+rent(?:s|ing)?\\s+)?(?:your|personal|user)\\s+(?:data|information)",
20 |        "transfer(?:s|red|ring)?\\s+(?:your|personal)\\s+data\\s+to\\s+(?:other\\s+entities|third\\s+parties|external)",
21 |        "commercial\\s+partners\\s+may\\s+(?:receive|access|use)\\s+your\\s+data",
22 |        "we\\s+reserve\\s+the\\s+right\\s+to\\s+share\\s+(?:aggregated|anonymized|de-identified|any)?\\s*(?:information|
23 |        "monetize(?:s|d|ing)?\\s+(?:user|personal)\\s+information",
24 |        "in\\s+the\\s+event\\s+of\\s+a\\s+merger(?:,|\\s+or)\\s+(?:acquisition|sale|bankruptcy).*data.*transferred"
25 |      ],
26 |      "keywords": [
27 |        "sell your data",
28 |        "share your information with third parties",
29 |        "disclose personal data to partners",
30 |        "monetize user information",
31 |        "share with advertisers"
32 |      ],
33 |      "explanation": "This clause allows the service to share, distribute, or sell your personal details with third-part
34 |      "why_it_matters": "Once your information is handed over to outside parties, you lose direct control over how it is
35 |      "recommendation": "Check if an opt-out mechanism is available or avoid submitting sensitive confidential data."
36 |    },
37 |    {
38 |      "id": "tracking",
39 |      "name": "Tracking & Telemetry",
40 |      "severity": "HIGH",
41 |      "weight": 14,
42 |      "color": "#ff3366",
43 |      "description": "Continuous device fingerprinting, behavioral tracking, cross-site telemetry, or keystroke monitori
44 |      "patterns": [
45 |        "track(?:s|ed|ing)?\\s+your\\s+(?:browsing|activity|behavior|location|online\\s+habits)",
46 |        "fingerprint(?:ing)?\\s+(?:your\\s+device|hardware|browser)",
47 |        "cross-site\\s+tracking",
48 |        "canvas\\s+fingerprinting",
49 |        "beacon(?:s)?|pixel\\s+tags?|web\\s+bugs?",
50 |        "record(?:s|ing)?\\s+(?:your\\s+screen|keystrokes|mouse\\s+movements|session\\s+replay)",
51 |        "monitor(?:s|ing)?\\s+(?:all|your)?\\s*(?:communications|browsing\\s+history|actions)",
52 |        "collect(?:s|ed|ing)?\\s+information\\s+about\\s+your\\s+activity\\s+across\\s+(?:third-party|other)\\s+websites
53 |      ],
54 |      "keywords": [
55 |        "cross-site tracking",
56 |        "device fingerprinting",
57 |        "session replay",
58 |        "keystroke recording",
59 |        "track your browsing activity"
60 |      ],
61 |      "explanation": "This clause enables active monitoring, device fingerprinting, or tracking of your online behavior
62 |      "why_it_matters": "Cross-site and hardware telemetry can build a persistent digital shadow profile of your habits,
63 |      "recommendation": "Use privacy-focused browsers, anti-fingerprinting shields, and strict tracker-blocking extensio
64 |    },
65 |    {
66 |      "id": "data_collection",
67 |      "name": "Data Collection",

```

```

68 |     "severity": "HIGH",
69 |     "weight": 12,
70 |     "color": "#ff3366",
71 |     "description": "Excessive collection of sensitive personal data, biometrics, contacts, communications, or device i
72 |     "patterns": [
73 |         "collect(?:s|ed|ing)?\\s+(?:biometric|health|financial|genetic|sensitive)\\s+(?:information|data)",
74 |         "access(?:es)?\\s+(?:to\\s+)?your\\s+(?:contacts|address\\s+book|microphone|camera|stored\\s+files|photos)",
75 |         "scrape(?:s|d|ing)?\\s+(?:data|information|content)",
76 |         "collect(?:s|ed|ing)?\\s+(?:all|any)\\s+information\\s+you\\s+(?:transmit|submit|upload|post|create)",
77 |         "gather(?:s|ed|ing)?\\s+(?:details|information)\\s+from\\s+external\\s+public\\s+and\\s+private\\s+records",
78 |         "including\\s+but\\s+not\\s+limited\\s+to\\s+social\\s+security|credit\\s+card|bank\\s+account"
79 |     ],
80 |     "keywords": [
81 |         "biometric data",
82 |         "access to your contacts",
83 |         "microphone and camera access",
84 |         "sensitive personal data",
85 |         "financial information"
86 |     ],
87 |     "explanation": "The provider reserves extensive rights to gather invasive categories of personal data, including c
88 |     "why_it_matters": "Over-collection expands your attack surface; if the provider suffers a data breach, your sensit
89 |     "recommendation": "Review permissions requested on your device and restrict sensor/contact access."
90 | },
91 | {
92 |     "id": "arbitration",
93 |     "name": "Mandatory Arbitration",
94 |     "severity": "HIGH",
95 |     "weight": 14,
96 |     "color": "#ff3366",
97 |     "description": "Forced binding arbitration and waiver of court trials and collective class action lawsuits.",
98 |     "patterns": [
99 |         "binding\\s+arbitration",
100 |         "waive(?:r|s|d)?\\s+(?:any\\s+right\\s+to\\s+)?(?:a\\s+)?jury\\s+trial",
101 |         "waive(?:r|s|d)?\\s+(?:any\\s+right\\s+to\\s+participate\\s+in\\s+a\\s+)?class\\s+action",
102 |         "class\\s+action\\s+waiver",
103 |         "disputes\\s+shall\\s+be\\s+resolved\\s+(?:exclusively\\s+)?(?:by|through)\\s+(?:confidential|individual|binding
104 |         "no\\s+class\\s+or\\s+representative\\s+actions?",
105 |         "agree\\s+not\\s+to\\s+sue\\s+in\\s+court"
106 |     ],
107 |     "keywords": [
108 |         "binding arbitration",
109 |         "class action waiver",
110 |         "waive jury trial",
111 |         "resolve disputes through arbitration",
112 |         "no class action"
113 |     ],
114 |     "explanation": "You surrender your constitutional right to take this company to court or participate in a joint co
115 |     "why_it_matters": "Arbitration proceedings are usually private, costly for individuals, and systematically favor c
116 |     "recommendation": "Check if an arbitration opt-out letter can be submitted within 30 days of registration."
117 | },
118 | {
119 |     "id": "unilateral_changes",
120 |     "name": "Unilateral Changes",
121 |     "severity": "HIGH",
122 |     "weight": 13,
123 |     "color": "#ff3366",
124 |     "description": "Changing legal terms or privacy policies at any time without notifying users or requiring fresh co
125 |     "patterns": [
126 |         "reserve\\s+the\\s+right\\s+to\\s+(?:modify|change|alter|update)\\s+(?:these\\s+terms|this\\s+agreement|policy)\\s+
127 |         "(?:modify|change|update)\\s+without\\s+(?:prior\\s+)?notice",
128 |         "continued\\s+use\\s+(?:of\\s+the\\s+service\\s+)?(?:constitutes|shall\\s+constitute|signifies)\\s+(?:your\\s+)?
129 |         "without\\s+obligation\\s+to\\s+notify\\s+you",
130 |         "effective\\s+immediately\\s+upon\\s+posting"
131 |     ],
132 |     "keywords": [
133 |         "modify without prior notice",
134 |         "at our sole discretion without notice",
135 |         "continued use constitutes acceptance",
136 |         "change these terms at any time"

```

```

137 |     ],
138 |     "explanation": "The company can alter policies, fees, or data handling rules at any time, and your continued usage",
139 |     "why_it_matters": "You could wake up to new invasive terms or higher subscription fees without ever receiving an e",
140 |     "recommendation": "Periodically review terms updates or monitor policy changes using change-detection tools."
141 | },
142 | {
143 |     "id": "liability_waiver",
144 |     "name": "Liability Waiver",
145 |     "severity": "MEDIUM",
146 |     "weight": 9,
147 |     "color": "#ffb700",
148 |     "description": "Total disclaimer of liability for data loss, service outages, bugs, or security vulnerabilities.",
149 |     "patterns": [
150 |         "provided\\s+[\\']?as\\s+is[\\']?\\s+and\\s+[\\']?as\\s+available[\\']?",
151 |         "disclaim(?:s)?\\s+(?:all|any)\\s+warranties",
152 |         "(?:under\\s+no\\s+circumstances\\s+shall|in\\s+no\\s+event\\s+shall)\\s+(?:the\\s+company|we|us|our\\s+affiliat",
153 |         "limit(?:ation)?\\s+of\\s+liability\\s+shall\\s+not\\s+exceed\\s+(?:the\\s+amount\\s+paid|\\$\\d+|one\\s+hundred",
154 |         "not\\s+liable\\s+for\\s+(?:any\\s+)?(?:data\\s+loss|security\\s+breaches|unauthorized\\s+access)"
155 |     ],
156 |     "keywords": [
157 |         "provided as is",
158 |         "in no event shall we be liable",
159 |         "disclaim all warranties",
160 |         "not liable for data loss or breach",
161 |         "limitation of liability"
162 |     ],
163 |     "explanation": "The company provides the product 'as-is' and completely disclaims legal responsibility if their so",
164 |     "why_it_matters": "If a critical security bug causes data corruption or identity theft, this clause shields the ve",
165 |     "recommendation": "Never rely solely on this vendor for sole data backups; maintain external independent copies."
166 | },
167 | {
168 |     "id": "auto_renewal",
169 |     "name": "Auto-Renewal & Recurring Billing",
170 |     "severity": "MEDIUM",
171 |     "weight": 8,
172 |     "color": "#ffb700",
173 |     "description": "Automatic recurring subscription renewals, often accompanied by strict cancellation timelines.",
174 |     "patterns": [
175 |         "auto(?:matically)?-?renew(?:s|ed|ing)?",
176 |         "renew(?:s|ed|ing)?\\s+automatically\\s+unless\\s+cancelled",
177 |         "recurring\\s+(?:billing|charge|subscription|fee)",
178 |         "charge(?:s|d)?\\s+your\\s+(?:payment\\s+method|credit\\s+card)\\s+without\\s+(?:additional|further)\\s+authoriz",
179 |         "cancel(?:ation)?\\s+at\\s+least\\s+\\d+\\s+days\\s+prior\\s+to\\s+(?:the\\s+end|renewal)",
180 |         "non-refundable\\s+(?:fees|charges|payments)"
181 |     ],
182 |     "keywords": [
183 |         "automatically renew",
184 |         "recurring subscription",
185 |         "cancel at least 24 hours prior",
186 |         "non-refundable fees",
187 |         "charge without further authorization"
188 |     ],
189 |     "explanation": "Your subscription will silently renew and charge your card automatically unless manually terminate",
190 |     "why_it_matters": "Users frequently get trapped in zombie billing cycles for unused services due to obscure cancel",
191 |     "recommendation": "Set calendar alerts 3-5 days ahead of billing cycles or use virtual credit cards with spend lim",
192 | },
193 | {
194 |     "id": "account_termination",
195 |     "name": "Account Termination",
196 |     "severity": "MEDIUM",
197 |     "weight": 8,
198 |     "color": "#ffb700",
199 |     "description": "Suspension or deletion of user accounts and content without prior notice, justification, or refund",
200 |     "patterns": [
201 |         "terminate\\s+(?:or\\s+suspend\\s+)?(?:your\\s+account|access)\\s+at\\s+(?:any\\s+time|our\\s+sole\\s+discretion",
202 |         "(?:terminate|suspend)\\s+without\\s+(?:prior\\s+)?notice\\s+(?:or\\s+liability)?",
203 |         "for\\s+any\\s+reason\\s+or\\s+(?:for\\s+)?no\\s+reason",
204 |         "forfeit(?:ure)?\\s+(?:of\\s+)?(?:all|any)\\s+(?:content|credits|funds|data)\\s+upon\\s+termination",
205 |         "no\\s+obligation\\s+to\\s+refund\\s+(?:unused\\s+)?fees"

```

```

206 | ],
207 | "keywords": [
208 |     "terminate at our sole discretion",
209 |     "suspend without notice",
210 |     "for any reason or no reason",
211 |     "forfeit content upon termination"
212 | ],
213 | "explanation": "The provider can ban your account, delete your assets, and block your access at will without explanation.",
214 | "why_it_matters": "You risk sudden loss of work, purchases, access to historical records, or communication channels.",
215 | "recommendation": "Regularly export and download backups of your cloud files, projects, and transactions."
216 | },
217 | {
218 |     "id": "third_party_access",
219 |     "name": "Third-Party Access",
220 |     "severity": "HIGH",
221 |     "weight": 11,
222 |     "color": "#ff3366",
223 |     "description": "Permitting outside integrations, ad networks, or vendors to access user accounts and content.",
224 |     "patterns": [
225 |         "third-party\\s+(?:service\\s+providers|vendors|contractors|integrations)\\s+may\\s+access",
226 |         "not\\s+responsible\\s+for\\s+(?:the\\s+privacy\\s+practices|actions|content)\\s+of\\s+third\\s+parties",
227 |         "allow(?:s)?\\s+third\\s+parties\\s+to\\s+(?:serve\\s+ads|collect\\s+data|place\\s+cookies)",
228 |         "integrated\\s+third-party\\s+services\\s+governed\\s+by\\s+their\\s+own\\s+terms",
229 |         "share\\s+with\\s+sub-processors\\s+without\\s+notification"
230 |     ],
231 |     "keywords": [
232 |         "third-party service providers may access",
233 |         "not responsible for third-party privacy",
234 |         "third parties to serve ads and collect",
235 |         "governed by third-party policies"
236 |     ],
237 |     "explanation": "External software vendors or third-party plug-ins can interface directly with your account or read your data.",
238 |     "why_it_matters": "Third-party supply chain vulnerabilities are a leading vector for corporate data leaks and identity theft.",
239 |     "recommendation": "Audit and disconnect unused connected apps and third-party account authorizations."
240 | },
241 | {
242 |     "id": "data_retention",
243 |     "name": "Indefinite Data Retention",
244 |     "severity": "MEDIUM",
245 |     "weight": 7,
246 |     "color": "#ffb700",
247 |     "description": "Retaining personal data, messages, or backups indefinitely even after account deletion.",
248 |     "patterns": [
249 |         "retain(?:s|ed|ing)?\\s+(?:your\\s+)?(?:data|information|content)\\s+(?:indefinitely|for\\s+an\\s+unlimited\\s+period)",
250 |         "may\\s+retain\\s+(?:archived|backup|residual)\\s+copies\\s+even\\s+after\\s+(?:you\\s+delete|account\\s+closure)",
251 |         "no\\s+obligation\\s+to\\s+delete\\s+(?:your\\s+)?(?:information|data)",
252 |         "retain\\s+and\\s+use\\s+your\\s+information\\s+to\\s+comply\\s+with\\s+legal\\s+obligations.*resolve\\s+disputes"
253 |     ],
254 |     "keywords": [
255 |         "retain data indefinitely",
256 |         "retain backup copies even after deletion",
257 |         "as long as we deem necessary",
258 |         "no obligation to delete"
259 |     ],
260 |     "explanation": "Deleting your account does not guarantee deletion of your personal records, which may be archived or stored elsewhere.",
261 |     "why_it_matters": "Zombie data retained forever creates a permanent target for future database breaches or subpoenas.",
262 |     "recommendation": "Exercise formal GDPR/CCPA 'Right to Erasure' requests where legally applicable."
263 | },
264 | {
265 |     "id": "location_tracking",
266 |     "name": "Location Tracking",
267 |     "severity": "HIGH",
268 |     "weight": 12,
269 |     "color": "#ff3366",
270 |     "description": "Harvesting precise GPS coordinates, Wi-Fi triangulation, cell tower IDs, or continuous background location data.",
271 |     "patterns": [
272 |         "collect(?:s|ed|ing)?\\s+(?:precise|exact|real-time|continuous)?\\s*(?:location|geo|location|gps)\\s+data",
273 |         "location\\s+(?:even\\s+when\\s+the\\s+app\\s+is\\s+closed|in\\s+the\\s+background)",
274 |         "track(?:s|ed|ing)?\\s+(?:your\\s+physical\\s+location|whereabouts|movements)",

```

```

275 |         "derive\\s+location\\s+from\\s+(?:ip\\s+address|wi-fi\\s+access\\s+points|cell\\s+towers|bluetooth)"
276 |     ],
277 |     "keywords": [
278 |         "precise geolocation data",
279 |         "location in the background",
280 |         "track your physical location",
281 |         "real-time location tracking"
282 |     ],
283 |     "explanation": "The app tracks or approximates your real-world geographic coordinates, potentially even when running in the background.",
284 |     "why_it_matters": "Real-time location records expose your residence, workplace, daily routines, and intimate personal information.",
285 |     "recommendation": "Set system location permissions to 'Only While Using App' or disable location entirely if not essential.",
286 | },
287 | {
288 |     "id": "advertising_profiling",
289 |     "name": "Advertising & Profiling",
290 |     "severity": "MEDIUM",
291 |     "weight": 8,
292 |     "color": "#ffb700",
293 |     "description": "Targeted behavioral ads, algorithmic psychological profiling, and interest-based segmentation.",
294 |     "patterns": [
295 |         "personalized\\s+(?:advertisements|ads|marketing|promotions)",
296 |         "interest-based\\s+advertising",
297 |         "create\\s+(?:a\\s+)?profile\\s+about\\s+you",
298 |         "target(?:ed)?\\s+advertising\\s+based\\s+on\\s+your\\s+(?:preferences|activity|browsing)",
299 |         "deliver\\s+relevant\\s+ads\\s+by\\s+analyzing\\s+your\\s+(?:interests|profile|demographics)"
300 |     ],
301 |     "keywords": [
302 |         "personalized advertisements",
303 |         "interest-based advertising",
304 |         "create a profile about you",
305 |         "targeted advertising based on"
306 |     ],
307 |     "explanation": "Your activity, searches, and demographic attributes are used to assemble an advertising profile for targeted marketing.",
308 |     "why_it_matters": "Targeted profiling can exploit behavioral vulnerabilities, shape opinions, and expose your privacy.",
309 |     "recommendation": "Opt out of personalized advertising within service settings and reset your advertising ID."
310 | },
311 | {
312 |     "id": "user_rights",
313 |     "name": "User Rights Surrender",
314 |     "severity": "HIGH",
315 |     "weight": 11,
316 |     "color": "#ff3366",
317 |     "description": "Surrendering intellectual property rights, granting irrevocable global content licenses, or moral rights waivers.",
318 |     "patterns": [
319 |         "grant(?:s)?\\s+(?:us\\s+)?(?:a\\s+)?(?:worldwide|perpetual|irrevocable|royalty-free|unrestricted)\\s+(?:license|agreement)",
320 |         "waive(?:s)?\\s+(?:all\\s+)?moral\\s+rights",
321 |         "you\\s+assign\\s+(?:all\\s+)?(?:ownership|intellectual\\s+property|rights)\\s+to\\s+us",
322 |         "we\\s+own\\s+all\\s+(?:feedback|submissions|content|ideas)\\s+provided\\s+by\\s+you",
323 |         "right\\s+to\\s+sublicense\\s+and\\s+commercialize\\s+your\\s+content"
324 |     ],
325 |     "keywords": [
326 |         "perpetual irrevocable royalty-free license",
327 |         "worldwide license to use and modify",
328 |         "waive moral rights",
329 |         "assign all intellectual property",
330 |         "sublicense and commercialize your content"
331 |     ],
332 |     "explanation": "You grant the company a permanent, global, royalty-free license to use, publish, monetize, or adapt your content without restriction.",
333 |     "why_it_matters": "You may lose commercial ownership or credit for creative works, files, or original designs uploaded to the platform.",
334 |     "recommendation": "Avoid storing or creating proprietary work or copyrighted media on platforms demanding broad IP licenses.",
335 | },
336 | {
337 |     "id": "cancellation_restrictions",
338 |     "name": "Cancellation Restrictions",
339 |     "severity": "LOW",
340 |     "weight": 5,
341 |     "color": "#38bdf8",
342 |     "description": "Complex hurdle-filled cancellation procedures, early termination penalties, or restricted contact with support.",
343 |     "patterns": [

```

```
344 |         "cancel(?:lation)?\\s+(?:must\\s+be\\s+made\\s+by|only\\s+by)\\s+(?:phone|written\\s+notice|certified\\s+mail)",
345 |         "early\\s+termination\\s+fee(?:s)?",
346 |         "penalt(?:y|ies)\\s+for\\s+(?:early\\s+)?cancellation",
347 |         "must\\s+provide\\s+(?:at\\s+least\\s+)?(?:30|60|90)\\s+days\\s+(?:prior\\s+)?notice\\s+to\\s+cancel",
348 |         "cannot\\s+be\\s+cancelled\\s+online"
349 |     ],
350 |     "keywords": [
351 |         "early termination fee",
352 |         "cancel only by phone",
353 |         "penalty for cancellation",
354 |         "certified mail to cancel"
355 |     ],
356 |     "explanation": "The provider imposes restrictive hurdles or fees to make terminating your account or contract deli
357 |     "why_it_matters": "Dark pattern cancellation flows prevent easy departure, forcing unneeded payments or unnecessar
358 |     "recommendation": "Check the exact cancellation procedures prior to providing payment details or signing long agre
359 | }
360 | ]
361 | };
```

File: frontend/js/app.js

Size: 10522 bytes

```

1 | /**
2 |  * CorruptX - Shared Application Logic
3 |  * Offline-first Architecture & Client Utilities
4 |  */
5 |
6 | const AppState = {
7 |   apiBase: "",
8 |   isBackendAvailable: false,
9 |   offlineMode: false
10 | };
11 |
12 | // SVG Icon Collection (Zero CDN dependencies)
13 | const ICONS = {
14 |   shield: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="cur
15 |   alertTriangle: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stro
16 |   checkCircle: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke
17 |   info: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="curre
18 |   externalLink: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" strok
19 |   copy: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="curre
20 |   trash: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="curre
21 |   search: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="cur
22 |   fileText: `<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="c
23 | };
24 |
25 | document.addEventListener("DOMContentLoaded", () => {
26 |   initMobileMenu();
27 |   detectBackendStatus();
28 |   highlightActiveNav();
29 | });
30 |
31 | /**
32 |  * Mobile navigation drawer
33 |  */
34 | function initMobileMenu() {
35 |   const toggleBtn = document.querySelector(".mobile-menu-btn");
36 |
37 |   const nav = document.querySelector(".main-nav");
38 |   if (toggleBtn && nav) {
39 |     toggleBtn.addEventListener("click", () => {
40 |       nav.classList.toggle("mobile-open");
41 |     });
42 |   }
43 | }
44 |
45 | /**
46 |  * Active navigation link highlighter
47 |  */
48 | function highlightActiveNav() {
49 |   const path = window.location.pathname;
50 |   const navLinks = document.querySelectorAll(".nav-link");
51 |   navLinks.forEach(link => {
52 |     const href = link.getAttribute("href");
53 |     if (href && (path.endsWith(href) || (href === "/" && (path === "/" || path.endsWith("index.html")))))
54 |       link.classList.add("active");
55 |   });
56 | }
57 |
58 | /**
59 |  * Detect whether the local Flask backend is reachable.
60 |  * If not, gracefully switch to the built-in offline engine.
61 |  */
62 | async function detectBackendStatus() {
63 |   const statusEl = document.getElementById("system-status-indicator");
64 |   const statusText = document.getElementById("system-status-text");
65 |
66 |   // Determine potential API roots
67 |   let candidates = ["/api/health"];

```

```

68 |   if (window.location.protocol === "file:" || window.location.port !== "5000") {
69 |       candidates.unshift("http://127.0.0.1:5000/api/health");
70 |   }
71 |
72 |   let healthy = false;
73 |   for (const url of candidates) {
74 |       try {
75 |           const res = await fetch(url, { method: "GET", mode: "cors", cache: "no-cache" });
76 |           if (res.ok) {
77 |               const data = await res.json();
78 |               if (data.status === "healthy") {
79 |                   AppState.isBackendAvailable = true;
80 |                   AppState.apiBase = url.replace("/api/health", "");
81 |                   healthy = true;
82 |                   break;
83 |               }
84 |           }
85 |       } catch (e) {
86 |           // Backend not running on this candidate
87 |       }
88 |   }
89 |
90 |   if (healthy) {
91 |       if (statusText) statusText.textContent = "ENGINE ONLINE (FLASK)";
92 |   } else {
93 |       AppState.isBackendAvailable = false;
94 |       AppState.offlineMode = true;
95 |       if (statusText) statusText.textContent = "OFFLINE ENGINE ACTIVE";
96 |       if (statusEl) {
97 |           statusEl.style.borderColor = "rgba(0, 242, 255, 0.4)";
98 |           statusEl.style.color = "var(--accent-cyan)";
99 |           const dot = statusEl.querySelector(".status-dot");
100 |           if (dot) {
101 |               dot.style.background = "var(--accent-cyan)";
102 |               dot.style.boxShadow = "0 0 8px var(--accent-cyan)";
103 |           }
104 |       }
105 |   }
106 | }
107 |
108 | /**
109 |  * Toast Notification System
110 |  */
111 | function showToast(message, type = "info") {
112 |     let container = document.getElementById("toast-container");
113 |     if (!container) {
114 |         container = document.createElement("div");
115 |         container.id = "toast-container";
116 |         document.body.appendChild(container);
117 |     }
118 |
119 |     const toast = document.createElement("div");
120 |     toast.className = `toast toast-${type}`;
121 |     toast.innerHTML = `
122 |         <span>${message}</span>
123 |     `;
124 |
125 |     container.appendChild(toast);
126 |     setTimeout(() => {
127 |         toast.style.opacity = "0";
128 |         toast.style.transform = "translateY(10px)";
129 |         setTimeout(() => toast.remove(), 300);
130 |     }, 3500);
131 | }
132 |
133 | /**
134 |  * LocalStorage Scan History & Active Scan Manager
135 |  * Includes memory fallback and auto-pruning to prevent QuotaExceededError
136 |  */

```

```

137 | const HISTORY_KEY = "corruptx_tc_scan_history";
138 | const CURRENT_SCAN_KEY = "corruptx_tc_active_scan";
139 |
140 | // In-memory global fallback to ensure scan is never lost during page transitions
141 | window.__CORRUPTX_ACTIVE_SCAN__ = null;
142 |
143 | function getScanHistory() {
144 |   try {
145 |     const raw = localStorage.getItem(HISTORY_KEY);
146 |     return raw ? JSON.parse(raw) : [];
147 |   } catch (e) {
148 |     console.error("Failed to read scan history:", e);
149 |     return [];
150 |   }
151 | }
152 |
153 | /**
154 |  * Creates a lightweight version of scan data for persistent history storage
155 |  */
156 | function createLightweightScanRecord(scanResult) {
157 |   // Prune large repetitive content to prevent LocalStorage quota overflow
158 |   const cleanResult = { ...scanResult };
159 |
160 |   // Keep only up to 50 clauses in all_clauses for history preview to keep size small
161 |   if (Array.isArray(cleanResult.all_clauses) && cleanResult.all_clauses.length > 50) {
162 |     cleanResult.all_clauses = cleanResult.all_clauses.slice(0, 50).map(c => ({
163 |       index: c.index,
164 |       snippet: (c.text || "").substring(0, 150) + "...",
165 |       is_flagged: c.is_flagged
166 |     }));
167 |   }
168 |
169 |   return {
170 |     id: "scan_" + Date.now() + "_" + Math.random().toString(36).substr(2, 5),
171 |     document_name: scanResult.document_name || "Submitted Policy",
172 |     analyzed_at: scanResult.analyzed_at || new Date().toISOString(),
173 |     risk_score: scanResult.risk_score,
174 |     risk_level: scanResult.risk_level,
175 |     clauses_analyzed: scanResult.clauses_analyzed,
176 |     red_flags: scanResult.red_flags,
177 |     severity_counts: scanResult.severity_counts,
178 |     full_data: cleanResult
179 |   };
180 | }
181 |
182 | function saveScanToHistory(scanResult) {
183 |   try {
184 |     const history = getScanHistory();
185 |     const record = createLightweightScanRecord(scanResult);
186 |
187 |     // Prepend new scan
188 |     history.unshift(record);
189 |     // Keep last 20 scans max
190 |     let trimmed = history.slice(0, 20);
191 |
192 |     try {
193 |       localStorage.setItem(HISTORY_KEY, JSON.stringify(trimmed));
194 |     } catch (quotaErr) {
195 |       console.warn("LocalStorage quota reached, pruning older history items...");
196 |       // Strip full_data from older items if quota is tight
197 |       trimmed = trimmed.slice(0, 5).map((item, idx) => {
198 |         if (idx > 0 && item.full_data) {
199 |           delete item.full_data.all_clauses;
200 |         }
201 |         return item;
202 |       });
203 |       localStorage.setItem(HISTORY_KEY, JSON.stringify(trimmed));
204 |     }
205 |     return record.id;

```

```
206 |     } catch (e) {
207 |         console.error("Failed to save scan history:", e);
208 |     }
209 | }
210 |
211 | function deleteScanHistoryItem(id) {
212 |     try {
213 |         const history = getScanHistory().filter(item => item.id !== id);
214 |         localStorage.setItem(HISTORY_KEY, JSON.stringify(history));
215 |     } catch (e) {
216 |         console.error("Failed to delete item:", e);
217 |     }
218 | }
219 |
220 | function clearScanHistory() {
221 |     try {
222 |         localStorage.removeItem(HISTORY_KEY);
223 |     } catch (e) {
224 |         console.error("Failed to clear history:", e);
225 |     }
226 | }
227 |
228 | function setActiveScan(scanResult) {
229 |     // Always update in-memory cache
230 |     window.__CORRUPTX_ACTIVE_SCAN__ = scanResult;
231 |     try {
232 |         sessionStorage.setItem(CURRENT_SCAN_KEY, JSON.stringify(scanResult));
233 |     } catch (e) {
234 |         console.warn("SessionStorage failed to set active scan, falling back to in-memory/localStorage:", e);
235 |         try {
236 |             // Try saving lightweight version if session storage full
237 |             const light = createLightweightScanRecord(scanResult).full_data;
238 |             sessionStorage.setItem(CURRENT_SCAN_KEY, JSON.stringify(light));
239 |         } catch (e2) {
240 |             console.warn("SessionStorage quota exceeded completely. Active scan preserved in memory.");
241 |         }
242 |     }
243 | }
244 |
245 | function getActiveScan() {
246 |     if (window.__CORRUPTX_ACTIVE_SCAN__) {
247 |         return window.__CORRUPTX_ACTIVE_SCAN__;
248 |     }
249 |     try {
250 |         const raw = sessionStorage.getItem(CURRENT_SCAN_KEY);
251 |         if (raw) {
252 |             const parsed = JSON.parse(raw);
253 |             window.__CORRUPTX_ACTIVE_SCAN__ = parsed;
254 |             return parsed;
255 |         }
256 |     } catch (e) {
257 |         console.error("Failed to load active scan from sessionStorage:", e);
258 |     }
259 |
260 |     // Fallback to latest history scan
261 |     const history = getScanHistory();
262 |     if (history.length > 0 && history[0].full_data) {
263 |         window.__CORRUPTX_ACTIVE_SCAN__ = history[0].full_data;
264 |         return history[0].full_data;
265 |     }
266 |
267 |     return null;
268 | }
```

File: frontend/js/scanner.js

Size: 9570 bytes

```

1  /**
2  * CorruptX - Scanner Page Controller
3  * Handles document intake, file drag/drop, sample loading,
4  * multi-step scan animation, and offline/online API dispatch.
5  */
6
7  document.addEventListener("DOMContentLoaded", () => {
8      const policyTextarea = document.getElementById("policy-text");
9      const docNameInput = document.getElementById("doc-name-input");
10     const charCountEl = document.getElementById("char-count");
11     const wordCountEl = document.getElementById("word-count");
12     const clauseEstimateEl = document.getElementById("clause-estimate");
13
14     const btnPaste = document.getElementById("btn-paste");
15     const btnClear = document.getElementById("btn-clear");
16     const btnLoadDemo = document.getElementById("btn-load-demo");
17     const btnScan = document.getElementById("btn-start-scan");
18
19     const fileInput = document.getElementById("file-upload-input");
20     const dropzone = document.getElementById("file-dropzone");
21
22     const scanningOverlay = document.getElementById("scanning-overlay");
23     const progressFill = document.getElementById("scan-progress-fill");
24
25     // Multi-step elements
26     const step1 = document.getElementById("step-1");
27     const step2 = document.getElementById("step-2");
28     const step3 = document.getElementById("step-3");
29     const step4 = document.getElementById("step-4");
30
31     // Check if URL has ?demo=1 or similar query parameter
32     const urlParams = new URLSearchParams(window.location.search);
33     if (urlParams.get("demo")) {
34         loadDemoPolicy(urlParams.get("demo") || "high_risk_social");
35     }
36
37     // Textarea input event
38     if (policyTextarea) {
39         policyTextarea.addEventListener("input", updateStats);
40         // Initial stats
41         updateStats();
42     }
43
44     function updateStats() {
45         const text = policyTextarea.value || "";
46         const chars = text.length;
47         const words = text.trim() ? text.trim().split(/\s+/).length : 0;
48         // Rough estimate of clauses (split on periods / newlines)
49         const clauses = text.trim() ? Math.max(1, text.split(/[.!?~\n]+/).filter(s => s.trim().length > 10).length) : 0;
50
51         if (charCountEl) charCountEl.textContent = chars.toLocaleString();
52         if (wordCountEl) wordCountEl.textContent = words.toLocaleString();
53         if (clauseEstimateEl) clauseEstimateEl.textContent = `~${clauses}`;
54     }
55
56     // Paste button
57     if (btnPaste) {
58         btnPaste.addEventListener("click", async () => {
59             try {
60                 if (navigator.clipboard && navigator.clipboard.readText) {
61                     const clipText = await navigator.clipboard.readText();
62                     if (clipText) {
63                         policyTextarea.value = clipText;
64                         updateStats();
65                         showToast("Policy text pasted from clipboard", "success");
66                         return;
67                     }
68                 }
69             } catch {
70                 // Clipboard not available
71             }
72         });
73     }
74
75     // File upload
76     if (fileInput) {
77         fileInput.addEventListener("change", async () => {
78             if (!fileInput.files || !fileInput.files.length) return;
79             const file = fileInput.files[0];
80             if (!file.type.startsWith("text/")) return;
81             const reader = new FileReader();
82             reader.onload = async () => {
83                 const text = reader.result;
84                 policyTextarea.value = text;
85                 updateStats();
86             };
87             reader.readAsText(file);
88         });
89     }
90
91     // Dropzone
92     if (dropzone) {
93         dropzone.addEventListener("dragover", (e) => {
94             e.preventDefault();
95             dropzone.classList.add("dragging");
96         });
97         dropzone.addEventListener("drop", async () => {
98             const items = Array.from(dropzone.querySelectorAll("div"));
99             const draggedItem = items.find(item => item.classList.contains("dragging"));
100             if (!draggedItem) return;
101             const text = draggedItem.textContent;
102             policyTextarea.value = text;
103             updateStats();
104             draggedItem.classList.remove("dragging");
105         });
106     }
107
108     // Demo button
109     if (btnLoadDemo) {
110         btnLoadDemo.addEventListener("click", () => {
111             loadDemoPolicy("high_risk_social");
112         });
113     }
114
115     // Clear button
116     if (btnClear) {
117         btnClear.addEventListener("click", () => {
118             policyTextarea.value = "";
119             updateStats();
120         });
121     }
122
123     // Scan button
124     if (btnScan) {
125         btnScan.addEventListener("click", () => {
126             const text = policyTextarea.value;
127             if (!text) return;
128             const docName = docNameInput.value;
129             if (!docName) return;
130             const scanProgress = document.getElementById("scan-progress");
131             const scanOverlay = document.getElementById("scan-overlay");
132             scanProgress.value = 0;
133             scanOverlay.classList.add("show");
134             // Simulate scanning process
135             const progressInterval = setInterval(() => {
136                 scanProgress.value += 10;
137             }, 100);
138             // Simulate API call
139             setTimeout(() => {
140                 clearInterval(progressInterval);
141                 scanOverlay.classList.remove("show");
142                 // Simulate successful scan
143                 const scanResult = {
144                     docName,
145                     charCount: text.length,
146                     wordCount: text.trim().split(/\s+/).length,
147                     clauseEstimate: `~${text.trim().split(/[.!?~\n]+/).length}`
148                 };
149                 // Store scan result
150                 localStorage.setItem("scanResult", JSON.stringify(scanResult));
151                 // Show success message
152                 showToast("Scan completed successfully", "success");
153             }, 2000);
154         });
155     }
156 }

```

```

68 |     }
69 |   } catch (e) {
70 |     // Clipboard permission denied or unsupported
71 |   }
72 |   policyTextarea.focus();
73 |   showToast("Please use Ctrl+V / Cmd+V to paste directly", "info");
74 | });
75 | }
76 |
77 | // Clear button
78 | if (btnClear) {
79 |   btnClear.addEventListener("click", () => {
80 |     policyTextarea.value = "";
81 |     docNameInput.value = "";
82 |     updateStats();
83 |     showToast("Cleared workspace", "info");
84 |   });
85 | }
86 |
87 | // Load Demo Policy
88 | if (btnLoadDemo) {
89 |   btnLoadDemo.addEventListener("click", () => {
90 |     loadDemoPolicy("high_risk_social");
91 |   });
92 | }
93 |
94 | function loadDemoPolicy(sampleId = "high_risk_social") {
95 |   let sample = null;
96 |   if (window.TC_SAMPLE_POLICIES.length > 0) {
97 |     sample = window.TC_SAMPLE_POLICIES.find(s => s.id === sampleId) || window.TC_SAMPLE_POLICIES[0];
98 |   }
99 |
100 |   if (sample) {
101 |     policyTextarea.value = sample.content;
102 |     if (docNameInput) docNameInput.value = sample.title;
103 |     updateStats();
104 |     showToast(`Loaded "${sample.title}" (${sample.expected_risk})`, "success");
105 |   } else {
106 |     showToast("Sample policy data not found", "error");
107 |   }
108 | }
109 |
110 | // File Upload Handlers
111 | if (fileInput) {
112 |   fileInput.addEventListener("change", (e) => {
113 |     const file = e.target.files[0];
114 |     if (file) handleUploadedFile(file);
115 |   });
116 | }
117 |
118 | if (dropzone) {
119 |   dropzone.addEventListener("click", () => fileInput.click());
120 |
121 |   ["dragenter", "dragover"].forEach(evt => {
122 |     dropzone.addEventListener(evt, (e) => {
123 |       e.preventDefault();
124 |       dropzone.classList.add("dragover");
125 |     });
126 |   });
127 |
128 |   ["dragleave", "drop"].forEach(evt => {
129 |     dropzone.addEventListener(evt, (e) => {
130 |       e.preventDefault();
131 |       dropzone.classList.remove("dragover");
132 |     });
133 |   });
134 |
135 |   dropzone.addEventListener("drop", (e) => {
136 |     const file = e.dataTransfer.files[0];

```

```
137 |     if (file) handleUploadedFile(file);
138 |   });
139 | }
140 |
141 | function handleUploadedFile(file) {
142 |   if (!file) return;
143 |
144 |   if (!file.name.match(/\.(txt|md|html|htm)$/i)) {
145 |     showToast("Please upload a .txt, .md, or .html file", "error");
146 |     return;
147 |   }
148 |
149 |   const reader = new FileReader();
150 |   reader.onload = (e) => {
151 |     let content = e.target.result || "";
152 |
153 |     // Strip basic HTML if html file
154 |     if (file.name.match(/\.(html|htm)$/i)) {
155 |       const parser = new DOMParser();
156 |       const doc = parser.parseFromString(content, "text/html");
157 |       // Remove scripts/styles
158 |       doc.querySelectorAll("script, style, noscript, meta").forEach(el => el.remove());
159 |       content = doc.body ? doc.body.innerText : content;
160 |     }
161 |
162 |     policyTextarea.value = content;
163 |     if (docNameInput) {
164 |       docNameInput.value = file.name.replace(/\.([^.]+)$/, "");
165 |     }
166 |     updateStats();
167 |     showToast(`File "${file.name}" loaded successfully`, "success");
168 |   };
169 |
170 |   reader.onerror = () => {
171 |     showToast("Error reading file", "error");
172 |   };
173 |
174 |   reader.readAsText(file);
175 | }
176 |
177 | // Start Scan Action
178 | if (btnScan) {
179 |   btnScan.addEventListener("click", () => {
180 |     startAnalysis();
181 |   });
182 | }
183 |
184 | async function startAnalysis() {
185 |   const text = policyTextarea.value ? policyTextarea.value.trim() : "";
186 |   const docName = (docNameInput.value && docNameInput.value.trim())
187 |     ? docNameInput.value.trim()
188 |     : "Analyzed Terms & Conditions";
189 |
190 |   if (!text) {
191 |     showToast("Please paste or upload Terms & Conditions text first.", "error");
192 |     policyTextarea.focus();
193 |     return;
194 |   }
195 |
196 |   if (text.length < 20) {
197 |     showToast("Input text is too short. Please paste full policy clauses.", "error");
198 |     return;
199 |   }
200 |
201 |   // Activate scanning overlay
202 |   if (scanningOverlay) {
203 |     scanningOverlay.classList.add("active");
204 |   }
205 | }
```

```

206 | // Step 1
207 | updateStepUI(1);
208 | if (progressFill) progressFill.style.width = "25%";
209 |
210 | try {
211 |     // Step 2 after small delay for animation
212 |     await sleep(350);
213 |     updateStepUI(2);
214 |     if (progressFill) progressFill.style.width = "50%";
215 |
216 |     let result = null;
217 |
218 |     // Try Backend if active, otherwise use client analyzer
219 |     if (AppState.isBackendAvailable && window.location.protocol !== "file:") {
220 |         try {
221 |             const apiEndpoint = (AppState.apiBase || "") + "/api/scan";
222 |             const res = await fetch(apiEndpoint, {
223 |                 method: "POST",
224 |                 headers: { "Content-Type": "application/json" },
225 |                 body: JSON.stringify({ text: text, document_name: docName })
226 |             });
227 |
228 |             if (res.ok) {
229 |                 result = await res.json();
230 |             } else {
231 |                 console.warn("Backend returned error, switching to client-side engine");
232 |             }
233 |         } catch (e) {
234 |             console.warn("Backend unreachable, switching to client-side engine:", e);
235 |         }
236 |     }
237 |
238 |     // If backend was unreachable or in offline mode, run client-side analyzer
239 |     if (!result) {
240 |         if (window.clientClauseAnalyzer) {
241 |             result = window.clientClauseAnalyzer.scan(text, docName);
242 |         } else {
243 |             throw new Error("Client analysis engine not initialized.");
244 |         }
245 |     }
246 |
247 |     // Step 3
248 |     await sleep(300);
249 |     updateStepUI(3);
250 |     if (progressFill) progressFill.style.width = "75%";
251 |
252 |     // Step 4
253 |     await sleep(250);
254 |     updateStepUI(4);
255 |     if (progressFill) progressFill.style.width = "100%";
256 |
257 |     // Save scan to sessionStorage for Results page
258 |     setActiveScan(result);
259 |     // Also save to scan history in localStorage
260 |     saveScanToHistory(result);
261 |
262 |     await sleep(300);
263 |
264 |     // Redirect to results page
265 |     // Handle both server routes (/results) and file protocol (results.html)
266 |     if (window.location.protocol === "file:" || window.location.pathname.endsWith(".html")) {
267 |         window.location.href = "results.html";
268 |     } else {
269 |         window.location.href = "/results";
270 |     }
271 |
272 | } catch (err) {
273 |     console.error("Scan execution failed:", err);
274 |     if (scanningOverlay) scanningOverlay.classList.remove("active");

```

```
275 |     showToast(`Scan failed: ${err.message || err}`, "error");
276 |   }
277 | }
278 |
279 | function updateStepUI(activeStepNum) {
280 |   const steps = [step1, step2, step3, step4];
281 |   steps.forEach((st, idx) => {
282 |     if (!st) return;
283 |     const stepIdx = idx + 1;
284 |     const iconBox = st.querySelector(".step-icon-indicator");
285 |
286 |     if (stepIdx < activeStepNum) {
287 |       st.className = "scan-step-item done";
288 |       if (iconBox) iconBox.innerHTML = ICONS.checkCircle;
289 |     } else if (stepIdx === activeStepNum) {
290 |       st.className = "scan-step-item active";
291 |       if (iconBox) iconBox.innerHTML = `<div class="step-spinner"></div>`;
292 |     } else {
293 |       st.className = "scan-step-item pending";
294 |       if (iconBox) iconBox.innerHTML = `•`;
295 |     }
296 |   });
297 | }
298 |
299 | function sleep(ms) {
300 |   return new Promise(resolve => setTimeout(resolve, ms));
301 | }
302 | });
```

File: frontend/js/results.js

Size: 16093 bytes

```

1 | /**
2 |  * CorruptX - Results Dashboard Controller
3 |  * Circular Risk Gauge, Filterable Red Flag Cards,
4 |  * Interactive Policy Syntax Highlighting, and Security Inspector Drawer.
5 |  */
6 |
7 | document.addEventListener("DOMContentLoaded", () => {
8 |     let scanData = getActiveScan();
9 |
10 |    // If no scan in sessionStorage, check history or fallback to demo
11 |    if (!scanData) {
12 |        const history = getScanHistory();
13 |        if (history.length > 0) {
14 |            scanData = history[0].full_data;
15 |        } else if (window.clientClauseAnalyzer && window.TC_SAMPLE_POLICIES && window.TC_SAMPLE_POLICIES.length) {
16 |            // Auto-scan the demo policy so the results page is never blank
17 |            const demo = window.TC_SAMPLE_POLICIES[0];
18 |            scanData = window.clientClauseAnalyzer.scan(demo.content, demo.title);
19 |            saveScanToHistory(scanData);
20 |            setActiveScan(scanData);
21 |        }
22 |    }
23 |
24 |    if (!scanData) {
25 |        window.location.href = "scanner.html";
26 |        return;
27 |    }
28 |
29 |    // Initialize UI components
30 |    renderOverview(scanData);
31 |    renderRedFlagCards(scanData);
32 |    renderPolicyHighlighter(scanData);
33 |    renderCategoryBreakdown(scanData);
34 |    initTabNavigation();
35 |    initActions(scanData);
36 | });
37 |
38 | /**
39 |  * Render Overview Banner and Circular Score Gauge
40 |  */
41 | function renderOverview(data) {
42 |     const docTitleEl = document.getElementById("res-doc-title");
43 |     const docDateEl = document.getElementById("res-doc-date");
44 |     const riskBadgeEl = document.getElementById("res-risk-badge");
45 |     const summaryTextEl = document.getElementById("res-summary-text");
46 |
47 |     const metricClausesEl = document.getElementById("metric-clauses");
48 |     const metricRedflagsEl = document.getElementById("metric-redflags");
49 |     const metricHighEl = document.getElementById("metric-high");
50 |     const metricMedEl = document.getElementById("metric-med");
51 |     const metricLowEl = document.getElementById("metric-low");
52 |
53 |     if (docTitleEl) docTitleEl.textContent = data.document_name || "Analyzed Document";
54 |     if (docDateEl) {
55 |         const dateStr = data.analyzed_at ? new Date(data.analyzed_at).toLocaleString() : new Date().toLocaleString();
56 |         docDateEl.textContent = `Scanned: ${dateStr}`;
57 |     }
58 |
59 |     // Risk Badge & Colors
60 |     const score = data.risk_score || 0;
61 |     const level = data.risk_level || "SAFE";
62 |
63 |     let badgeClass = "badge-safe";
64 |     let gaugeColor = "var(--accent-green)";
65 |
66 |     if (level === "HIGH RISK") {
67 |         badgeClass = "badge-high";

```

```

68 |     gaugeColor = "var(--accent-crimson)";
69 | } else if (level === "CAUTION") {
70 |     badgeClass = "badge-medium";
71 |     gaugeColor = "var(--accent-amber)";
72 | }
73 |
74 | if (riskBadgeEl) {
75 |     riskBadgeEl.className = `badge ${badgeClass} risk-level-badge`;
76 |     riskBadgeEl.textContent = level;
77 | }
78 |
79 | if (summaryTextEl) {
80 |     summaryTextEl.textContent = data.summary || "";
81 | }
82 |
83 | // Metrics counts
84 | if (metricClausesEl) metricClausesEl.textContent = data.clauses_analyzed || 0;
85 | if (metricRedflagsEl) metricRedflagsEl.textContent = data.red_flags || 0;
86 | if (metricHighEl) metricHighEl.textContent = data.severity_counts?.HIGH || 0;
87 | if (metricMedEl) metricMedEl.textContent = data.severity_counts?.MEDIUM || 0;
88 | if (metricLowEl) metricLowEl.textContent = data.severity_counts?.LOW || 0;
89 |
90 | // Animate Circular Gauge
91 | animateCircularGauge(score, gaugeColor);
92 | }
93 |
94 | function animateCircularGauge(score, strokeColor) {
95 |     const fillPath = document.getElementById("gauge-fill-path");
96 |     const scoreText = document.getElementById("gauge-score-value");
97 |
98 |     if (!fillPath || !scoreText) return;
99 |
100 |     const radius = 80;
101 |     const circumference = 2 * Math.PI * radius;
102 |     fillPath.style.strokeDasharray = `${circumference}`;
103 |
104 |     // Start with full offset (empty circle)
105 |     fillPath.style.strokeDashoffset = `${circumference}`;
106 |     fillPath.style.stroke = strokeColor;
107 |
108 |     // Trigger animation after slight timeout
109 |     setTimeout(() => {
110 |         const targetOffset = circumference - (score / 100) * circumference;
111 |         fillPath.style.strokeDashoffset = `${targetOffset}`;
112 |
113 |         // Counter animation
114 |         let current = 0;
115 |         const stepTime = 1200 / Math.max(score, 1);
116 |         const counter = setInterval(() => {
117 |             if (current >= score) {
118 |                 scoreText.textContent = score;
119 |                 clearInterval(counter);
120 |             } else {
121 |                 current++;
122 |                 scoreText.textContent = current;
123 |             }
124 |         }, Math.max(10, stepTime));
125 |     }, 100);
126 | }
127 |
128 | /**
129 |  * Render Detected Red Flag Cards
130 |  */
131 | function renderRedFlagCards(data) {
132 |     const container = document.getElementById("red-flags-container");
133 |     if (!container) return;
134 |
135 |     const flagged = data.flagged_clauses || [];
136 |

```

```

137 | // Update tab counter badge
138 | const flagsCountBadge = document.getElementById("tab-count-flags");
139 | if (flagsCountBadge) flagsCountBadge.textContent = flagged.length;
140 |
141 | if (flagged.length === 0) {
142 |     container.innerHTML = `
143 |         <div class="card" style="text-align: center; padding: 48px 24px;">
144 |             <div style="color: var(--accent-green); margin-bottom: 12px;">${ICONS.checkCircle}</div>
145 |             <h3 style="color: #fff; margin-bottom: 8px;">No Critical Red Flags Detected</h3>
146 |             <p style="max-width: 480px; margin: 0 auto;">This document does not contain clauses triggering high-sever
147 |         </div>
148 |     `;
149 |     return;
150 | }
151 |
152 | container.innerHTML = flagged.map((flag, i) => {
153 |     const severityLower = flag.severity.toLowerCase();
154 |     return `
155 |         <div class="flag-card flag-card-${severityLower}" data-severity="${flag.severity}" id="flag-card-${flag.index}">
156 |             <div class="flag-card-header">
157 |                 <div class="flag-category-title">
158 |                     <span class="category-name-text">${escapeHtml(flag.category_name)}</span>
159 |                     <span class="badge badge-${severityLower}">${flag.severity} RISK</span>
160 |                 </div>
161 |                 <div class="flag-card-actions">
162 |                     <button class="btn btn-outline btn-sm btn-inspect-clause" data-clause-index="${flag.index}">
163 |                         Inspect in Policy
164 |                     </button>
165 |                 </div>
166 |             </div>
167 |
168 |             <div class="clause-quote-box">
169 |                 "${escapeHtml(flag.text)}"
170 |             </div>
171 |
172 |             <div class="flag-analysis-body">
173 |                 <div class="analysis-callout">
174 |                     <div class="callout-header">
175 |                         ${ICONS.info}
176 |                         <span>What this clause means</span>
177 |                     </div>
178 |                     <div class="callout-text">
179 |                         ${escapeHtml(flag.explanation)}
180 |                     </div>
181 |                 </div>
182 |
183 |                 <div class="analysis-callout">
184 |                     <div class="callout-header" style="color: var(--accent-crimson);">
185 |                         ${ICONS.alertTriangle}
186 |                         <span>Why this matters</span>
187 |                     </div>
188 |                     <div class="callout-text">
189 |                         ${escapeHtml(flag.why_it_matters)}
190 |                     </div>
191 |                 </div>
192 |             </div>
193 |
194 |             <div class="recommendation-bar">
195 |                 ${ICONS.checkCircle}
196 |                 <div><strong>Suggested Action:</strong> ${escapeHtml(flag.recommendation)}</div>
197 |             </div>
198 |         `;
199 |     }).join("");
200 |
201 |
202 | // Attach click listeners to "Inspect in Policy" buttons
203 | container.querySelectorAll(".btn-inspect-clause").forEach(btn => {
204 |     btn.addEventListener("click", () => {
205 |         const idx = parseInt(btn.getAttribute("data-clause-index"), 10);

```

```

206 |     switchToHighlighterTab(idx);
207 | });
208 | });
209 |
210 | // Filter handlers
211 | initFilters(flagged);
212 | }
213 |
214 | function initFilters(allFlags) {
215 |     const filterBtns = document.querySelectorAll(".filter-btn");
216 |     filterBtns.forEach(btn => {
217 |         btn.addEventListener("click", () => {
218 |             filterBtns.forEach(b => { b.classList.remove("active");
219 |             btn.classList.add("active");
220 |
221 |             const filter = btn.getAttribute("data-filter");
222 |             const cards = document.querySelectorAll(".flag-card");
223 |
224 |             cards.forEach(card => {
225 |                 const sev = card.getAttribute("data-severity");
226 |                 if (filter === "all" || sev === filter) {
227 |                     card.style.display = "block";
228 |                 } else {
229 |                     card.style.display = "none";
230 |                 }
231 |             });
232 |         });
233 |     });
234 | }
235 |
236 | /**
237 |  * Render Interactive Policy Analysis (Full Text with Clause Highlighting)
238 |  */
239 | function renderPolicyHighlighter(data) {
240 |     const docContainer = document.getElementById("policy-full-text-container");
241 |     if (!docContainer) return;
242 |
243 |     const allClauses = data.all_clauses || [];
244 |
245 |     if (allClauses.length === 0) {
246 |         docContainer.textContent = "No document text available.";
247 |         return;
248 |     }
249 |
250 |     docContainer.innerHTML = allClauses.map(clause => {
251 |         let highlightClass = "highlight-safe";
252 |         if (clause.is_flagged) {
253 |             if (clause.severity === "HIGH") highlightClass = "highlight-high";
254 |             else if (clause.severity === "MEDIUM") highlightClass = "highlight-medium";
255 |             else if (clause.severity === "LOW") highlightClass = "highlight-low";
256 |         }
257 |
258 |         return `<span class="clause-highlight ${highlightClass}" data-clause-index="${clause.index}" title="${clause.is_f
259 |     }).join("");
260 |
261 |     // Attach click events on highlighted spans
262 |     docContainer.querySelectorAll(".clause-highlight").forEach(span => {
263 |         span.addEventListener("click", () => {
264 |             const idx = parseInt(span.getAttribute("data-clause-index"), 10);
265 |             const clauseObj = allClauses[idx];
266 |             inspectClause(clauseObj, span);
267 |         });
268 |     });
269 | }
270 |
271 | function inspectClause(clause, targetSpan) {
272 |     // Highlight currently inspected span
273 |     document.querySelectorAll(".clause-highlight").forEach(el => { el.classList.remove("active-inspected");
274 |     if (targetSpan) targetSpan.classList.add("active-inspected");

```

```

275 |
276 | const pane = document.getElementById("inspector-content-pane");
277 | const placeholder = document.getElementById("inspector-placeholder");
278 |
279 | if (!pane || !placeholder) return;
280 |
281 | placeholder.style.display = "none";
282 | pane.style.display = "flex";
283 |
284 | const sevLower = (clause.severity || "safe").toLowerCase();
285 |
286 | pane.innerHTML = `
287 |   <div style="display: flex; align-items: center; justify-content: space-between;">
288 |     <span class="badge badge-${sevLower}">${clause.severity} RISK</span>
289 |     <span style="font-family: var(--font-mono); font-size: 0.78rem; color: var(--text-dim);">Clause #${clause.in
290 |   </div>
291 |
292 |   <div>
293 |     <h4 style="font-size: 1.05rem; color: #fff; margin-bottom: 4px;">${escapeHtml(clause.category_name || "Stand
294 |     <div style="font-size: 0.8rem; color: var(--text-dim); font-family: var(--font-mono);">
295 |       Taxonomy Risk Weight: ${clause.weight || 0} pts
296 |     </div>
297 |   </div>
298 |
299 |   <div style="background: rgba(8, 12, 20, 0.8); border: 1px solid var(--border-card); border-radius: 6px; padding:
300 |     "${escapeHtml(clause.text)}"
301 |   </div>
302 |
303 |   <div class="analysis-callout" style="padding: 10px;">
304 |     <div class="callout-header">${ICONS.info} <span>Plain English</span></div>
305 |     <div class="callout-text" style="font-size: 0.84rem;">${escapeHtml(clause.explanation)}</div>
306 |   </div>
307 |
308 |   <div class="analysis-callout" style="padding: 10px;">
309 |     <div class="callout-header" style="color: var(--accent-crimson);">${ICONS.alertTriangle} <span>Why it
310 |     <div class="callout-text" style="font-size: 0.84rem;">${escapeHtml(clause.why_it_matters)}</div>
311 |   </div>
312 |
313 |   <div class="recommendation-bar" style="padding: 8px 12px; font-size: 0.82rem;">
314 |     <div><strong>Action:</strong> ${escapeHtml(clause.recommendation)}</div>
315 |   </div>
316 | `;
317 | }
318 |
319 | function switchToHighlighterTab(clauseIndex) {
320 |   const highlighterTabBtn = document.querySelector('[data-tab="highlighter"]');
321 |   if (highlighterTabBtn) highlighterTabBtn.click();
322 |
323 |   setTimeout(() => {
324 |     const span = document.querySelector(`.clause-highlight[data-clause-index="${clauseIndex}"]`);
325 |     if (span) {
326 |       span.scrollIntoView({ behavior: "smooth", block: "center" });
327 |       span.click();
328 |     }
329 |   }, 150);
330 | }
331 |
332 | /**
333 |  * Render Category Risk Breakdown Chart
334 |  */
335 | function renderCategoryBreakdown(data) {
336 |   const container = document.getElementById("category-breakdown-container");
337 |   if (!container) return;
338 |
339 |   const categories = data.categories || [];
340 |
341 |   if (categories.length === 0) {
342 |     container.innerHTML = `<p style="text-align: center; color: var(--text-muted); padding: 32px;">No risk categor
343 |   return;

```

```

344 |   }
345 |
346 |   const maxCount = Math.max(...categories.map(c => c.count), 1);
347 |
348 |   container.innerHTML = `
349 |     <div class="breakdown-list">
350 |       ${categories.map(cat => {
351 |
352 |         const percent = Math.min(100, Math.round((cat.count / maxCount) * 100));
353 |         const sevLower = cat.severity.toLowerCase();
354 |         let color = "var(--accent-blue)";
355 |         if (cat.severity === "HIGH") color = "var(--accent-crimson)";
356 |         else if (cat.severity === "MEDIUM") color = "var(--accent-amber)";
357 |
358 |         return `
359 |           <div class="breakdown-item">
360 |             <div class="breakdown-item-header">
361 |               <div class="breakdown-category-label">
362 |                 <span>${escapeHtml(cat.category_name)}</span>
363 |                 <span class="badge badge-${sevLower}">${cat.severity}</span>
364 |               </div>
365 |               <div class="breakdown-counts">
366 |                 <span>${cat.count} clause${cat.count > 1 ? 's' : ''} flagged</span>
367 |               </div>
368 |               <div class="breakdown-track">
369 |                 <div class="breakdown-fill" style="width: ${percent}%; background: ${color}; box-shadow: 0 0 10px ${color}>
370 |                 </div>
371 |               </div>
372 |             </div>
373 |           </div>
374 |         </div>
375 |       `;
376 |     }
377 |
378 | /**
379 |  * Tab Navigation (Red Flags vs Highlighter vs Breakdown)
380 |  */
381 | function initTabNavigation() {
382 |   const tabs = document.querySelectorAll(".tab-btn");
383 |   const views = {
384 |     cards: document.getElementById("view-red-flags"),
385 |     highlighter: document.getElementById("view-policy-analysis"),
386 |     breakdown: document.getElementById("view-category-breakdown")
387 |   };
388 |
389 |   tabs.forEach(tab => {
390 |     tab.addEventListener("click", () => {
391 |       tabs.forEach(t => t.classList.remove("active"));
392 |       tab.classList.add("active");
393 |
394 |       const target = tab.getAttribute("data-tab");
395 |       Object.keys(views).forEach(k => {
396 |         if (views[k]) {
397 |           views[k].style.display = (k === target) ? "block" : "none";
398 |         }
399 |       });
400 |     });
401 |   });
402 | }
403 |
404 | /**
405 |  * Export & Action handlers
406 |  */
407 | function initActions(data) {
408 |   const btnExport = document.getElementById("btn-export-report");
409 |   const btnCopy = document.getElementById("btn-copy-summary");
410 |   const btnRescan = document.getElementById("btn-rescan-new");
411 |
412 |   if (btnExport) {

```

```

413 |     btnExport.addEventListener("click", () => {
414 |         const blob = new Blob([JSON.stringify(data, null, 2)], { type: "application/json" });
415 |         const url = URL.createObjectURL(blob);
416 |         const a = document.createElement("a");
417 |         a.href = url;
418 |         a.download = `corruptx_tc_report_${Date.now()}.json`;
419 |         a.click();
420 |         URL.revokeObjectURL(url);
421 |         showToast("Security report exported as JSON", "success");
422 |     });
423 | }
424 |
425 | if (btnCopy) {
426 |     btnCopy.addEventListener("click", () => {
427 |         const summaryMarkdown = `
428 | # CORRUPTX T&C SECURITY REPORT
429 | **Target:** ${data.document_name}
430 | **Risk Score:** ${data.risk_score}/100 [${data.risk_level}]
431 | **Analyzed Clauses:** ${data.clauses_analyzed}
432 | **Red Flags Identified:** ${data.red_flags}
433 |
434 | ## Critical Red Flags:
435 | ${((data.flagged_clauses || []).map(f => `- **${f.category_name} (${f.severity})**: "${f.text.substring(0, 100)}..."` -
436 |
437 | *Generated by CorruptX T&C Red Flag Scanner (Cybersecurity Engine)*
438 |     `).trim());
439 |
440 |         navigator.clipboard.writeText(summaryMarkdown).then(() => {
441 |             showToast("Executive summary copied to clipboard", "success");
442 |         }).catch(() => {
443 |             showToast("Clipboard copy failed", "error");
444 |         });
445 |     });
446 | }
447 |
448 | if (btnRescan) {
449 |     btnRescan.addEventListener("click", () => {
450 |         window.location.href = "scanner.html";
451 |     });
452 | }
453 | }
454 |
455 | function escapeHtml(text) {
456 |     if (!text) return "";
457 |     return text
458 |         .replace(/&/g, "&amp;")
459 |         .replace(/</g, "&lt;")
460 |         .replace(/>/g, "&gt;")
461 |         .replace(/"/g, "&quot;")
462 |         .replace(/'/g, "&#039;");
463 | }

```

File: frontend/js/history.js

Size: 6276 bytes

```

1 | /**
2 |  * CorruptX - Scan History Controller
3 |  * Manages persistent client-side scan logs with localStorage,
4 |  * restoration into Results dashboard, deletion, and export.
5 |  */
6 |
7 | document.addEventListener("DOMContentLoaded", () => {
8 |   renderHistoryView();
9 |   initHistoryControls();
10 | });
11 |
12 | function renderHistoryView() {
13 |   const history = getScanHistory();
14 |   const listContainer = document.getElementById("history-list-container");
15 |   const emptyState = document.getElementById("history-empty-state");
16 |
17 |   const statTotalScans = document.getElementById("stat-total-scans");
18 |   const statAvgScore = document.getElementById("stat-avg-score");
19 |   const statHighRiskScans = document.getElementById("stat-high-risk-scans");
20 |
21 |   if (!listContainer) return;
22 |
23 |   // Compute metrics
24 |   const total = history.length;
25 |   const avg = total > 0 ? Math.round(history.reduce((acc, h) => acc + (h.risk_score || 0), 0) / total) : 0;
26 |   const highRiskCount = history.filter(h => h.risk_level === "HIGH RISK").length;
27 |
28 |   if (statTotalScans) statTotalScans.textContent = total;
29 |   if (statAvgScore) statAvgScore.textContent = `${avg}/100`;
30 |   if (statHighRiskScans) statHighRiskScans.textContent = highRiskCount;
31 |
32 |   if (total === 0) {
33 |     if (emptyState) emptyState.style.display = "block";
34 |     listContainer.innerHTML = "";
35 |     return;
36 |   }
37 |
38 |   if (emptyState) emptyState.style.display = "none";
39 |
40 |   listContainer.innerHTML = history.map(item => {
41 |     const sevLower = (item.risk_level || "SAFE").toLowerCase().replace(" ", "-");
42 |     let badgeClass = "badge-safe";
43 |     if (item.risk_level === "HIGH RISK") badgeClass = "badge-high";
44 |     else if (item.risk_level === "CAUTION") badgeClass = "badge-medium";
45 |
46 |     const dateFormatted = new Date(item.analyzed_at).toLocaleString();
47 |
48 |     return `
49 |       <div class="card history-card" id="hist-item-${item.id}" style="margin-bottom: 16px; padding: 20px;">
50 |         <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 16px;">
51 |           <div style="display: flex; align-items: center; gap: 16px;">
52 |             <div style="width: 48px; height: 48px; border-radius: 10px; background: rgba(15, 23, 42, 0.9); border: 1px solid #fff;">
53 |               <span style="font-size: 1.15rem; font-weight: 800; color: #fff; line-height: 1;">${item.risk_score}</span>
54 |             <span style="font-size: 0.6rem; color: var(--text-dim); font-family: var(--font-mono);">/${item.risk_level}</span>
55 |           </div>
56 |           <div>
57 |             <h3 style="font-size: 1.05rem; font-weight: 700; color: #fff; margin-bottom: 4px;">${escapeHtml(item.title)}
58 |             <div style="display: flex; align-items: center; gap: 12px; font-size: 0.8rem; color: var(--text-dim); font-family: var(--font-mono);">
59 |               <span>${dateFormatted}</span>
60 |               <span>•</span>
61 |               <span>${item.clauses_analyzed} Clauses</span>
62 |               <span>•</span>
63 |               <span>${item.red_flags} Red Flags</span>
64 |             </div>
65 |           </div>
66 |         </div>
67 |       `

```

```

68 |         <div style="display: flex; align-items: center; gap: 12px;">
69 |             <span class="badge ${badgeClass}">${item.risk_level}</span>
70 |             <button class="btn btn-outline btn-sm btn-view-scan" data-id="${item.id}">
71 |                 View Analysis
72 |             </button>
73 |             <button class="btn btn-danger btn-sm btn-delete-scan" data-id="${item.id}" title="Delete Record">
74 |                 ${ICONS.trash}
75 |             </button>
76 |         </div>
77 |     </div>
78 | </div>
79 | `;
80 | }).join("");
81 |
82 | // Attach event handlers
83 | listContainer.querySelectorAll(".btn-view-scan").forEach(btn => {
84 |     btn.addEventListener("click", () => {
85 |         const id = btn.getAttribute("data-id");
86 |         const record = history.find(h => h.id === id);
87 |         if (record && record.full_data) {
88 |             setActiveScan(record.full_data);
89 |             if (window.location.protocol === "file:" || window.location.pathname.endsWith(".html")) {
90 |                 window.location.href = "results.html";
91 |             } else {
92 |                 window.location.href = "/results";
93 |             }
94 |         }
95 |     });
96 | });
97 |
98 | listContainer.querySelectorAll(".btn-delete-scan").forEach(btn => {
99 |     btn.addEventListener("click", () => {
100 |         const id = btn.getAttribute("data-id");
101 |         deleteScanHistoryItem(id);
102 |         showToast("Scan record removed", "info");
103 |         renderHistoryView();
104 |     });
105 | });
106 | }
107 |
108 | function initHistoryControls() {
109 |     const btnClear = document.getElementById("btn-clear-history");
110 |     const btnExportAll = document.getElementById("btn-export-all-history");
111 |     const btnLoadSampleHistory = document.getElementById("btn-load-sample-history");
112 |
113 |     if (btnClear) {
114 |         btnClear.addEventListener("click", () => {
115 |             if (confirm("Are you sure you want to clear all local scan history?")) {
116 |                 clearScanHistory();
117 |                 renderHistoryView();
118 |                 showToast("Scan history cleared", "info");
119 |             }
120 |         });
121 |     }
122 |
123 |     if (btnExportAll) {
124 |         btnExportAll.addEventListener("click", () => {
125 |             const history = getScanHistory();
126 |             if (history.length === 0) {
127 |                 showToast("No scan records to export", "error");
128 |                 return;
129 |             }
130 |             const blob = new Blob([JSON.stringify(history, null, 2)], { type: "application/json" });
131 |             const url = URL.createObjectURL(blob);
132 |             const a = document.createElement("a");
133 |             a.href = url;
134 |             a.download = `corruptx_history_${Date.now()}.json`;
135 |             a.click();
136 |             URL.revokeObjectURL(url);

```

```
137 |     showToast("Full history exported", "success");
138 |   });
139 | }
140 |
141 | if (btnLoadSampleHistory) {
142 |   btnLoadSampleHistory.addEventListener("click", () => {
143 |     if (window.clientClauseAnalyzer && window.TC_SAMPLE_POLICIES) {
144 |       window.TC_SAMPLE_POLICIES.forEach(sample => {
145 |         const scan = window.clientClauseAnalyzer.scan(sample.content, sample.title);
146 |         saveScanToHistory(scan);
147 |       });
148 |       renderHistoryView();
149 |       showToast("Loaded sample scan history", "success");
150 |     }
151 |   });
152 | }
153 | }
154 |
155 | function escapeHtml(text) {
156 |   if (!text) return "";
157 |   return text
158 |     .replace(/&/g, "&amp;")
159 |     .replace(/</g, "&lt;")
160 |     .replace(/>/g, "&gt;")
161 |     .replace(/"/g, "&quot;")
162 |     .replace(/'/g, "&#039;");
163 | }
```

File: frontend/js/sample_data.js

Size: 6986 bytes

```

1 | /**
2 |  * Preloaded Hackathon Demo Policies
3 |  * Allows zero-network instant testing of the scanner.
4 |  */
5 | window.TC_SAMPLE_POLICIES = [
6 |   {
7 |     "id": "high_risk_social",
8 |     "title": "HyperNet Social App Terms & Privacy Policy",
9 |     "expected_risk": "HIGH RISK",
10 |    "description": "Aggressive mobile app with tracking, third-party data sales, unilateral modifications, and forced an
11 |    "content": "Welcome to HyperNet. By accessing or using our services, mobile application, or websites, you agree to b
12 |   },
13 |   {
14 |     "id": "moderate_risk_saas",
15 |     "title": "NovaDesk Cloud Workspace Agreement",
16 |     "expected_risk": "CAUTION",
17 |     "description": "A business SaaS platform with commercial auto-renewal, standard liability caps, unilateral maintenar
18 |     "content": "NovaDesk Cloud Services Master Subscription Agreement.\n\n1. SCOPE AND SERVICES\nNovaDesk provides onlin
19 |   },
20 |   {
21 |     "id": "safe_open_source",
22 |     "title": "AegisGuard Privacy-First Client Terms",
23 |     "expected_risk": "SAFE",
24 |     "description": "Open-source zero-knowledge utility with end-to-end encryption, zero tracking, and no third-party sha
25 |     "content": "AegisGuard Terms of Use and Privacy Charter.\n\n1. ZERO KNOWLEDGE ARCHITECTURE\nAegisGuard operates as a
26 |   }
27 | ];

```